Rapport sur l'Extension Trigonométrique

Groupe GL49

24 janvier 2025

Table des matières

1 Introduction **3**

2 Présentation Générale **3**
2.1 Contexte du Projet . . . 3
2.2 Objectifs de l'Extension . . . 3
2.3 Architecture Globale . . . 4

3 Réduction Modulaire **4**
3.1 Stockage des flottants et erreurs associées . . . 4
3.2 Solutions envisagées . . . 5
3.3 Modulo classique . . . 5
3.3.1 Explication de l'algorithme . . . 5
3.3.2 Analyse théorique . . . 5
3.3.3 Algorithme . . . 6
3.3.4 Test de précision . . . 6
3.3.5 Test de rapidité . . . 7
3.4 Réduction itérative . . . 7
3.4.1 Explication de l'algorithme . . . 7
3.4.2 Analyse théorique . . . 7
3.4.3 Algorithme . . . 8
3.4.4 Test de précision . . . 8
3.4.5 Test de rapidité . . . 8
3.5 Méthode de Réduction d'argument . . . 9
3.5.1 Explication de l'algorithme [3] . . . 9
3.5.2 Analyse théorique . . . 9
3.5.3 Algorithme . . . 10
3.5.4 Test de précision . . . 10
3.5.5 Test de rapidité . . . 10
3.5.6 Validation et choix de l'algorithme . . . 11
3.6 Choix final . . . 12

4 Fonctions trigonométriques **12**
4.1 Algorithmes de calcul . . . 12
4.2 Traitement des entrées . . . 12
4.3 Sinus . . . 13
4.3.1 Propriétés du sinus . . . 13

4.3.2 Choix des algorithmes . . . 13
4.3.3 Développement en série de Taylor de Sinus . . . 14
4.3.4 Test de précision . . . 14
4.3.5 Test de rapidité . . . 18
4.4 Cosinus . . . 18
4.4.1 Propriétés du cosinus . . . 18
4.4.2 Choix des algorithmes . . . 19
4.4.3 Développement en série de Taylor de Cosinus . . . 19
4.4.4 Tests de précision . . . 19
4.4.5 Test de rapidité . . . 22
4.5 Tangente . . . 22
4.5.1 Propriétés de la tangente . . . 23
4.5.2 Choix des algorithmes . . . 23
4.5.3 Tests de Précision . . . 23
4.5.4 Test de rapidité . . . 25
4.6 Arcsinus . . . 26
4.6.1 Propriétés de l'arcsinus . . . 26
4.6.2 Choix des algorithmes . . . 26
4.6.3 Développement en série de Taylor de l'arcsinus . . . 26
4.6.4 Tests de Précision . . . 27
4.6.5 Tests de Rapidité . . . 28
4.7 Arccosinus . . . 29
4.7.1 Propriétés de l'arccosinus . . . 29
4.7.2 Choix des algorithmes . . . 30
4.7.3 Tests de Précision . . . 30
4.7.4 Tests de Rapidité . . . 31
4.8 Arccosinus . . . 32
4.8.1 Propriétés de l'arccosinus . . . 32
4.8.2 Choix des algorithmes . . . 33
4.8.3 Tests de Précision . . . 33
4.8.4 Tests de Rapidité . . . 34
4.8.5 Analyse des tests . . . 35
4.9 Arctangente . . . 35
4.9.1 Propriétés de l'arctangente . . . 35
4.9.2 Choix des algorithmes . . . 36
4.9.3 Développement en série de Taylor de l'arctangente . . . 36
4.9.4 Tests de Précision . . . 37
4.9.5 Tests de Rapidité . . . 38
4.9.6 Analyse des tests . . . 39

5 Validation des tests **39**

6 Conclusion **40**

7 Sources **40**

1 Introduction

Dans le cadre de l'extension trigonométrique, ce rapport vise à retracer l'implémentation d'un bibliothèque "math.deca" chargée de procurer les fonctions trigonométriques usuelles. Ce document n'est pas un manuel utilisateur, un document dédié à cela est disponible sous le nom de Manuel utilisateur.

Les extensions trigonométriques jouent un rôle crucial dans divers domaines scientifiques et techniques, notamment en facilitant des calculs complexes et en optimisant les performances des applications mathématiques.

L'objectif principal de ce document est de fournir une vue d'ensemble claire et détaillée du contexte du projet, des méthodologies adoptées, ainsi que des résultats obtenus. Nous abordons les problématiques rencontrées et les solutions mises en oeuvre.

Ce rapport est structuré comme suit : une présentation générale du projet, une description détaillée de l'implémentation des différentes parties avec pour chacune de ces dernières une partie résultats et performances. Le choix à été fait d'inclure ces dimensions en tant que sous parties car elles ont influé sur les choix faits. De plus les tokens tels que [x] renverront vers des sources ayant servis dans les paragraphes où ils se situent.

2 Présentation Générale

2.1 Contexte du Projet

Le projet d'extension trigonométrique s'inscrit dans un contexte où ces fonctions surprésentes dans le milieu informatique, physique et mathématique sont des algorithmes appelés via des bibliothèques sans se poser plus de questions. Oui ces algorithmes sont efficaces mais ce projet permet de se poser la question de comment ont été implémentées ces dernières, à quels enjeux ces implémentations ont du faire face et quelles solutions ont alors été envisagées.

Les principaux enjeux identifiés sont les suivants :

— La nécessité d'avoir des outils précis malgrès l'environnement de travail reposant sur des calculs avec des flottants

— Le besoin d'avoir des algorithmes efficaces permettant des calculs sur de grandes bases de données

Ce projet vise donc à répondre à ces besoins en proposant une solution robuste et performante, adaptée à une utilsation variée d'un potentiel utilisateur du language deca.

2.2 Objectifs de l'Extension

Les objectifs principaux de l'extension trigonométrique se déclinent comme suit :

— **Implémentation de fonctions trigonométriques :** Développer les fonctions trigonométriques suivantes :
  — Cosinus
  — Sinus
  — Tangente
  — Arccosinus
  — Arcsinus

  - Arctangente
- **Précision :** S'assurer que les calculs réalisés par ces extensions atteignent un niveau de précision élevé, essentiel pour garantir des résultats fiables. Pour étudier cela une fonction **ULP** sera implémentée, elle permettera de donner le "Unit Last Place", c'est à dire l'unité du dernier bit d'une variable afin de connaître sa précision.
- **Optimisation des performances :** Concevoir des algorithmes efficaces, capables de traiter des volumes de données importants tout en réduisant le temps de calcul.
- **Compatibilité :** Développer des outils facilement intégrables à un environnement deca.
- **Validation et test :** Mettre en place des scénarios de test robustes pour valider les performances et la fiabilité des implémentations.

L'atteinte de ces objectifs permettra de valider notre bibliothèque math.deca.

2.3 Architecture Globale

Le codage de cette bibliothèque a été décomposée en 3 étapes distinctes :

- Réduction Modulaire
- Algorithmes de réduction d'intervalle
- Algorithmes de calcul de valeurs

Pour chaque étape, les méthodes utilisées seront décrites dans un premier temps puis une mise en oeuvre de leur implémentation et enfin leur test.
Pour effectuer nos test, nous avons décidé de transposer notre code dans le language Java afin de pouvoir l'executer sur une machine. L'ensemble du code écrit dans le fichier math.java est le même que celui de notre bibliothèque math.deca. Executer nos tests sur cet environnement nous permet aussi de pouvoir utiliser des bibliothèques ayant des outils graphiques afin de visualiser et analyser nos résultats plus simplement.

3 Réduction Modulaire

Le premier défi rencontré en réflechissant sur l'implémentation des fonctions sinus et cosinus à été d'utiliser la périodicité de ces fonctions pour pourvoir réduire dans un premier temps le domaine d'étude à [-2pi ;2pi]. Cependant, on manipule ici pas uniquement des entiers mais des float qui sont la manière de stocker nos variables d'entrée. Le problème principal des calculs de flottants est qu'ils ne représentent pas des valeurs exactes mais des approxmations. L'enjeu était alors de minimiser au maximum la propagation de ces erreurs lors de la réduction modulaire.

3.1 Stockage des flottants et erreurs associées

Le stockage des nombres flottants suit la norme IEEE 754[1][2], qui permet de représenter sur 32 bits un sous-ensemble des réels en utilisant une combinaison d'un signe (1 bit), d'un exposant biaisé (8 bits), et d'une mantisse (23 bits). Un nombre flottant est défini par l'équation $(-1)^s \times 2^{e-127} \times (1 + m)$, où s est le bit de signe, e est l'exposant avec un biais de 127 (pour le format simple précision), et m représente la mantisse. Par exemple, pour le nombre 1.0, les valeurs de ses composants sont : $s = 0$, $e = 127$, et

$m = 0$. La précision absolue est alors définie par l'ULP (*Unit of Least Precision*), qui pour 1.0 est $2^{-23} \approx 1.192 \times 10^{-7}$.

Cependant, cette représentation engendre plusieurs limitations :

— **Approximation et erreur d'arrondi :** Tous les réels ne pouvant pas être représentés exactement, les calculs flottants sont sujets à des erreurs d'arrondi. Par exemple, le flottant immédiatement inférieur à 1.0 est $1.0 - 2^{-24}$.

— **Répartition inégale :** La densité des flottants est très variable. Une forte densité est observée autour de 0, où les nombres représentés sont très rapprochés, tandis que dans les grands entiers, l'écart entre flottants successifs devient plus significatif (par exemple, autour de 10^9, cet écart est de l'ordre de 128).

— **Non-associativité des calculs :** En raison des erreurs d'arrondi, les opérations arithmétiques ne respectent pas toujours les propriétés classiques. Par exemple, $(1.1 + 1.1) \neq 2.2$ en calcul flottant.

Ces limitations nécessitent une attention particulière dans le choix des algorithmes et dans l'analyse de la précision. Par exemple, pour notre extension, il est impératif de maintenir une précision relative au bit près mais on comprends alors que ce dernier ne correspond pas à un même ordre de précision en fonction de l'ordre de grandeur de nos valeurs. Il faut aussi optimiser les algorithmes pour minimiser les erreurs d'arrondi.

3.2 Solutions envisagées

Le choix est alors le suivant : étudier différents algorithmes en théorie et en pratique sur différentes plages de valeurs avec des tests de précision et de rapidité. Voici les trois algorithmes retenus :

— Modulo bibliothèque standard

— Réduction itérative

— Scaling

3.3 Modulo classique

3.3.1 Explication de l'algorithme

Le fonctionnement peut être résumé ainsi : la valeur initiale x est divisée par "modulo" pour obtenir le quotient. Ensuite, seule la partie fractionnaire de ce quotient est conservée. Cette partie fractionnaire est ensuite re-multipliée par "modulo", ce qui ramène la valeur dans l'intervalle $[-modulo, modulo]$.

3.3.2 Analyse théorique

En termes de complexité, cette méthode est très rapide, car elle repose sur une seule division et une seule multiplication, suivies d'une soustraction. Ces opérations sont généralement très efficaces, même pour des valeurs de grande amplitude, car les processeurs modernes sont optimisés pour ces instructions.

Cependant, la précision de cette méthode est limitée par les erreurs inhérentes aux nombres flottants. La division et la soustraction impliquent des arrondis, surtout lorsque la valeur de x est très grande. Par exemple, si x est de l'ordre de 10^{12}, l'erreur relative peut atteindre 10^{-6}.

L'algorithme de modulo classique en simple précision (32 bits) introduit des erreurs dues à la précision limitée des nombres flottants, définie par une mantisse de 23 bits, soit une erreur relative d'environ $2^{-23} \approx 1.192 \times 10^{-7}$ pour des modulo classiques de l'ordre de 10^0. Cette erreur relative croît proportionnellement à la valeur de l'entrée x dans le calcul $x \mod modulo$, car la division par "modulo" amplifie les arrondis. Pour déterminer la limite à laquelle l'erreur dépasse une tolérance fixée par exemple à 0.01, on résoud :

$$x \times 2^{-23} < 0.01 \implies x < \frac{0.01}{2^{-23}}.$$

En remplaçant 2^{-23} par sa valeur approximative, on obtient :

$$x < \frac{0.01}{1.192 \times 10^{-7}} \approx 83,886.$$

Ainsi, pour des valeurs $x > 8*10^3$, l'erreur absolue dépasse 0.01, ce qui rend cette méthode assez imprécise rapidement pour des valeurs de x relativement grandes.

3.3.3 Algorithme

Listing 1 – Methode de réduction classique avec modulo

3.3.4 Test de précision

Pour procéder à un test de précision, nous devions pourvoir avoir une valeur de référence. Le choix à été d'utiliser l'opportunité qu'offre le language Java avec la manipulation de réels double précision (mantisse de 52 bits). On compare alors notre valeur obtenue par la fonction qu'on veut tester avec la valeur rendue par le modulo classique mais sur des réels double précision. Pour pourvoir nous représenter les erreurs en fonction de l'ordre de grandeur des réels en entrée, nous comparons sur 1000 valeurs aléatoires de cet ordre de grandeur et traçons l'erreur moyenne en fonction de l'ordre de grandeur des réels. Le choix à été ici fait de tester avec un modulo de 2Pi car cela représente d'une part un modulo que l'on va utiliser mais aussi une valeur non entière de l'ordre de 10^0.

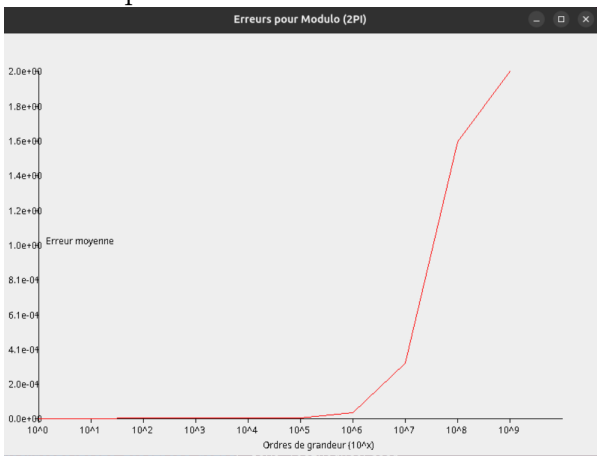


FIGURE 1 – Erreurs jusqu'à 10^9

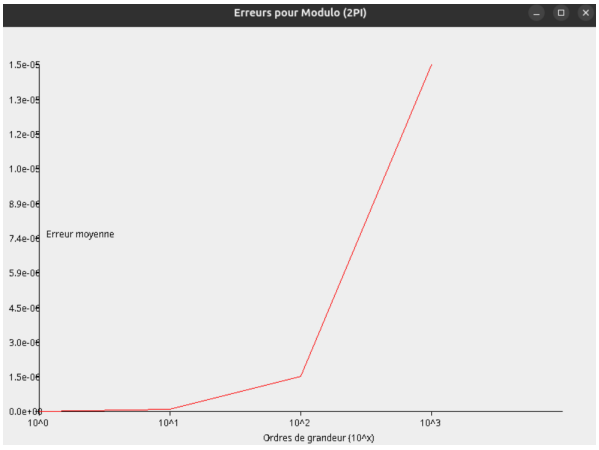


FIGURE 2 – Erreurs jusqu'à 10^3

On observe ici comme notre analyse théorique le prévoyait que la précision du modulo classique décroit quand les réels deviennent trop grands et à partir de 10^3 on tombe à une précision faible de l'ordre de 10^{-4}.

3.3.5 Test de rapidité

Pour tester la rapidité de nos algorithmes, il a été tracé à l'aide d'outils graphiques disponibles sur Java le temps d'execution que prends une méthode pour plusieurs quantités de données à différents ordres de grandeur.

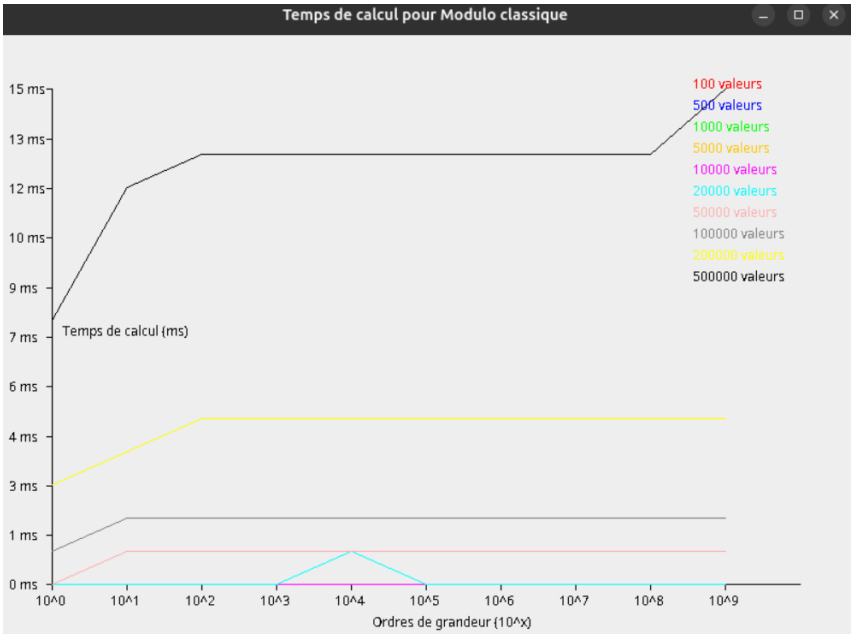


FIGURE 3 – Rapidité Modulo classique

On constate bien une augmentation du temps de caclul lorsque plus de valeurs sont calculées mais pas de différences quand les données d'entrées sont d'un ordre de grandeur plus élevé.

3.4 Réduction itérative

3.4.1 Explication de l'algorithme

L'algorithme de réduction itérative consiste à ramener une valeur x dans l'intervalle $[-modulo, modulo]$ en ajoutant ou en soustrayant de manière répétée des multiples de modulo connus (ex : modulo, 10*modulo, 100*modulo). Cet algorithme a pour but d'éviter les erreurs accrues par les divisions et multiplications du modulo classique.

3.4.2 Analyse théorique

En termes de complexité, la méthode itérative est moins performante que le modulo classique, car elle repose sur des soustractions ou additions répétées, ce qui peut devenir coûteux pour des valeurs de x très éloignées de l'intervalle cible. Cependant, elle offre une meilleure précision théorique, car elle évite la division flottante, qui amplifie les erreurs liées à la mantisse limitée des nombres flottants.

L'erreur principale de cette méthode est liée à l'accumulation des erreurs d'arrondi à chaque étape. On s'attend donc a une précision moindre pour des grandes valeurs mais aussi une rapidité assez faible.

3.4.3 Algorithme

Listing 2 – Méthode de réduction itérative

3.4.4 Test de précision

On utilise la même procédure pour tester cet algorithme.

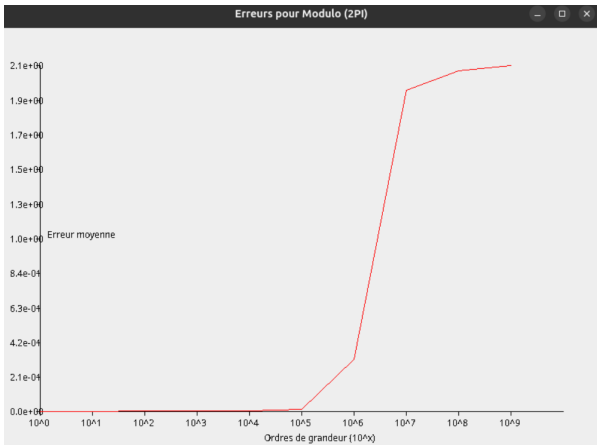


FIGURE 4 – Erreurs jusqu'à 10^9

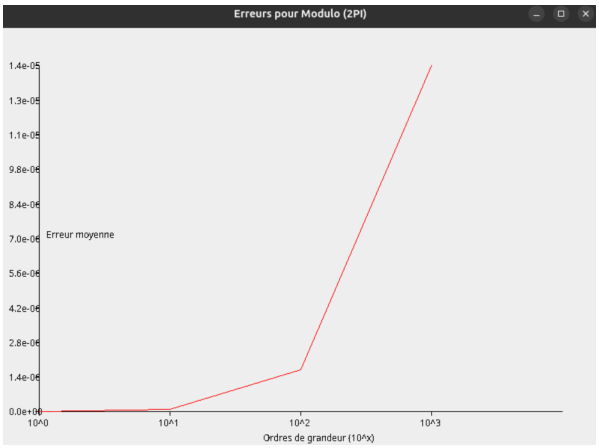


FIGURE 5 – Erreurs jusqu'à 10^3

Les résultats montrent que la méthode itérative est peu précise dès que les valeurs grandissent. Pour un ordre de grandeur de 10^9 on est à une erreur de l'ordre de 10^0. C'est aux alentours de 10^3 que l'erreur commence à devenir significative.

3.4.5 Test de rapidité

Pour tester la rapidité de nos algorithmes, il a été tracé à l'aide d'outils graphiques disponibles sur Java le temps d'execution que prends une méthode pour plusieurs quantités de données à différents ordres de grandeur.

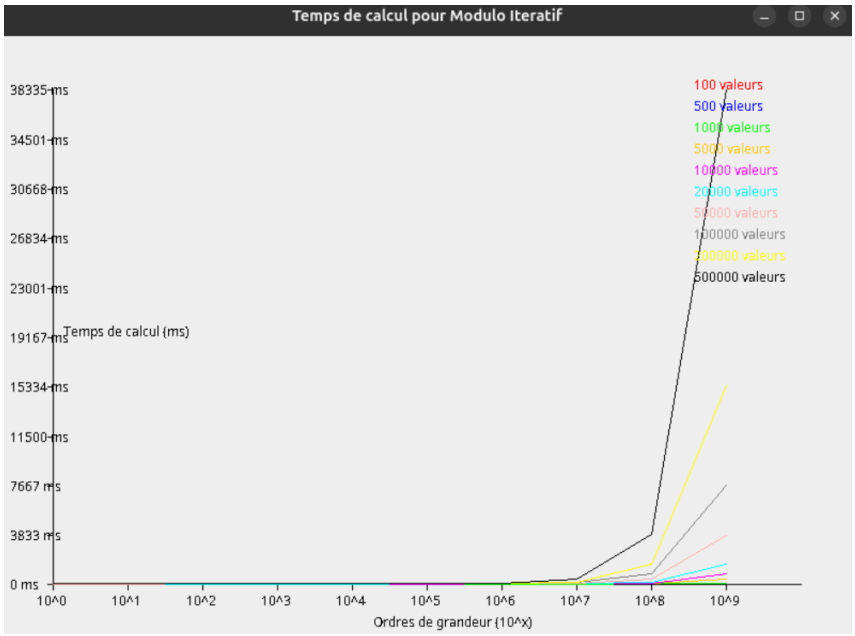


FIGURE 6 – Rapidité Modulo itératif

On constate bien une augmentation du temps de caclul lorsque plus de valeurs sont calculées mais aussi et surtout un temps de calcul augmentant fortement avec l'augmentation de l'ordre de grandeur des données d'entrées jusqu'à arriver à l'ordre de la seconde pour des puissances de 10 proche de 9.

3.5 Méthode de Réduction d'argument

3.5.1 Explication de l'algorithme [3]

L'algorithme de scaling repose sur une approche mathématique consistant à ramener une valeur x dans l'intervalle $[-modulo, modulo]$ en divisant x par $2modulo$ pour obtenir une valeur normalisée. Ensuite, seule la partie fractionnaire du quotient est conservée. Cette fraction est multipliée par $2modulo$ pour ramener la valeur dans la plage cible.

En d'autres termes, cet algorithme procède ainsi : 1. Diviser x par $2modulo$ pour normaliser la valeur dans la plage $[k, k+1]$, où k est un entier. 2. Extraire la partie fractionnaire du quotient pour isoler la valeur dans un intervalle réduit. 3. Re-multiplier cette fraction par $2modulo$ pour ramener la valeur dans $[-modulo, modulo]$.

Ce procédé évite les répétitions successives comme dans la méthode itérative, tout en réduisant les erreurs associées au modulo classique, car il repose sur des divisions et des multiplications précises plutôt que des soustractions.

3.5.2 Analyse théorique

En termes de complexité, cette méthode est très efficace car elle effectue un nombre constant d'opérations (une division, une extraction de fraction, et une multiplication). Elle est donc indépendante de la magnitude de x, ce qui en fait une solution bien adaptée aux grands réels. De plus, elle est plus précise que le modulo classique, car elle évite les arrondis répétés lors de soustractions.

Cependant, les erreurs peuvent toujours provenir de deux sources principales :
-Les divisions impliquent des approximations qui dépendent de la précision de la mantisse (23 bits en simple précision ou 52 bits en double précision).

-EL'extraction de la fraction repose sur la précision des flottants, et des écarts peuvent apparaître pour des valeurs très grandes.

3.5.3 Algorithme

Voici l'implémentation en Java de l'algorithme de scaling :

Listing 3 – Méthode de réduction par Scaling

3.5.4 Test de précision

On utilise la même procédure pour tester cet algorithme.

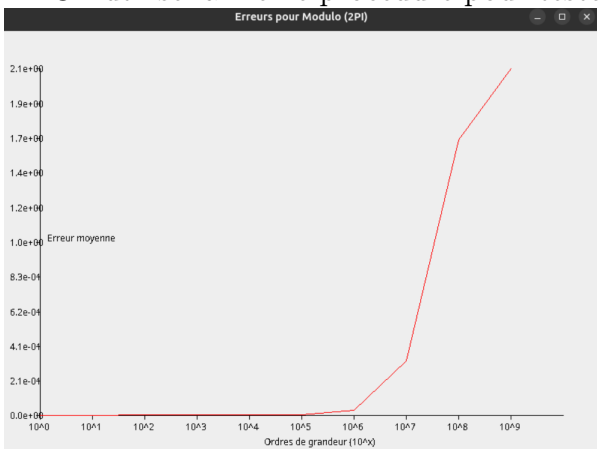


FIGURE 7 – Erreurs jusqu'à 10^9

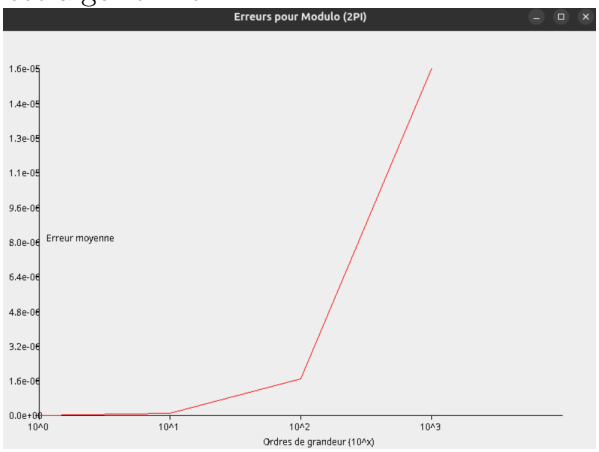


FIGURE 8 – Erreurs jusqu'à 10^3

On observe une assez bonne précision jusqu'à 10^3 l'erreur reste de l'ordre de 10^{-5}. Mais elle croit tout de même jusqu'à 10^0 aux environs de 10^9 ce qui est bien trop grand pour avoir des valeurs fiables.

3.5.5 Test de rapidité

Pour tester la rapidité de nos algorithmes, il a été tracé à l'aide d'outils graphiques disponibles sur Java le temps d'execution que prends une méthode pour plusieurs quantités de données à différents ordres de grandeur.

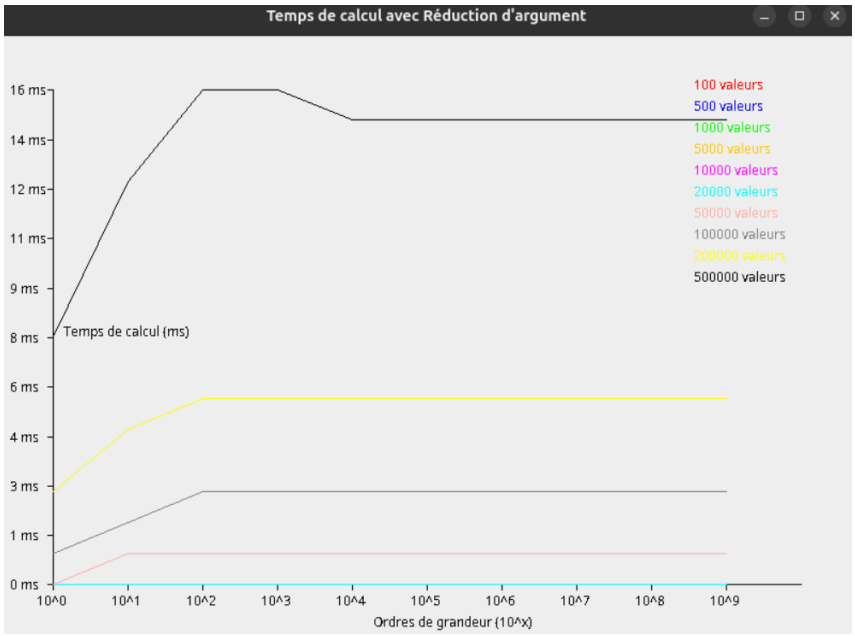


FIGURE 9 – Rapidité Réduction d'argument

On constate bien une augmentation du temps de caclul lorsque plus de valeurs sont calculées mais pas de différences quand les données d'entrées sont d'un ordre de grandeur plus élevé.

3.5.6 Validation et choix de l'algorithme

On peut alors comparer les différents algorithmes comme ci dessous en terme d'erreurs :

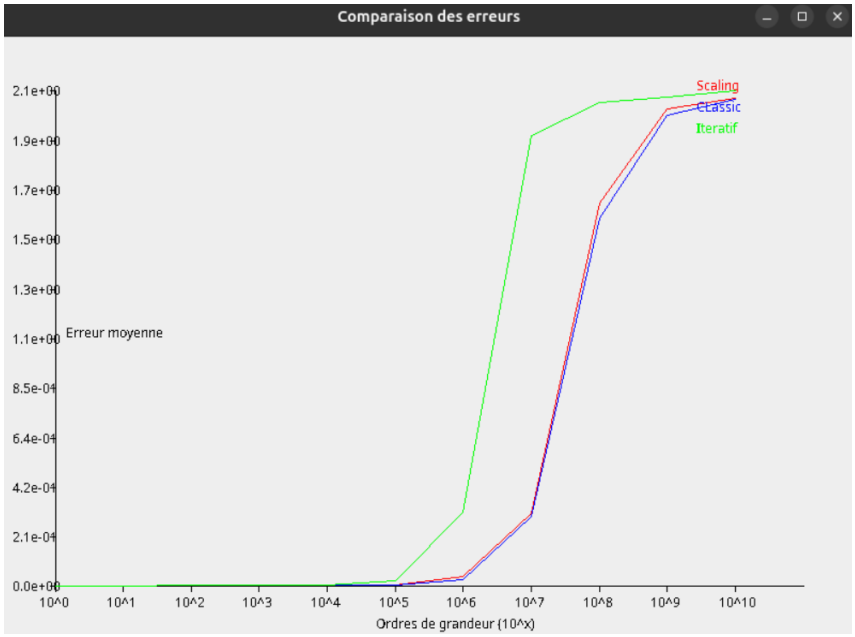


FIGURE 10 – Comparaison des erreurs

On voit alors qu'une différence faible entre l'algorithme de "Réduction d'argument" et celui du "modulo classique". On peut par contre mettre à l'écart l'algorithme itératif qui se voit être moins précis que les deux autres. On peut expliquer cela par l'addition des

erreurs des float dus aux nombreuses itérations ce qui ne permet pas d'être plus précis que les méthodes utilisant la division et multiplication finalement.

On peut départager alors ces deux algorithmes avec leurs temps de calcul. On retrouve en réalité des résultats similaires quoiqu'un tout petit peu plus rapides pour le modulo classique.

3.6 Choix final

Le choix a alors été fait de garder l'utilisation du **modulo classique** car plus rapide et cela permet aussi d'avoir un risque de moins sur la polyvalence de notre implémentation car l'opérateur modulo à été grandement testé au fil du temps ! On relève en revanche que pour des grandes valeurs on arrive à des erreurs moyennes proche de 2 pour une réduction modulaire sur l'intervalle [0 ;2PI] ce qui revient autant à prendre une valeur aléatoire dans cet intervalle. Il faut alors entrevoir des limites à notre bibliothèque math.deca quand l'ordre de grandeur des entrées est trop élevé. Le choix est alors de refuser les ordres de grandeurs supérieurs à 10^5 et donc de garantir une réduction modulaire précise à 10^{-5}.

4 Fonctions trigonométriques

Maintenant qu'un outil de réduction modulaire est disponible, nous avons tout les outils nécessaires à nos algorithmes.

4.1 Algorithmes de calcul

Dans un premier temps notre approche des méthodes trigonométriques s'est reposée sur la recherche d'algorithmes dans le but d'exhiber leurs forces et faiblesses. Le but de cette méthode était de pouvoir diviser nos domaines d'études en orientant nos valeurs d'entrées dans des algorithmes spécialisés dans ces plages de valeurs. Pour les algorithmes de calcul de sinus et cosinus, les algorithmes suivants ont été envisagés :

— **Séries de Taylor[4] :** Ces dernières sont une approche mathématique assez simple mais qui demandent potentiellement des calculs gourmands pour une bonne précision lorsqu'on s'éloigne de 1

— **Approximation d'une fonction par les polynômes de Tchebychev [5] :** Cette méthode permet d'approximer une fonction par des polynômes dit de "Tchebychev". Cette méthode a une précision inchangée sur un intervalle donné

— **Algorithme de Cordic[6] :** Cet algorithme est celui majoritairement embarqué dans nos calculatrices, il évite les multiplications et divisions ce qui lui permet d'être précis et facilement embarquable dans des systèmes légers.

Les choix d'implémentations de ces algorithmes ou non seront justifiés dans les sections dédiées à l'implémentation de chaque fonction.

4.2 Traitement des entrées

Nos algorithmes respecteront le même schéma afin de calculer les résultats :

— Vérification de l'argument, appartient t'il au domaine de définition

— Si oui on utilise les méthodes de réduction modulaire en se servant des périodicité des fonctions

— On appelle une fonction qui vérifie si notre valeur est une valeur particulière, si oui on renvoie immédiatement le résultat précalculé

— Sinon on utilise les symétries pour réduire l'intervalle

— On appelle les algorithmes de calcul

Ces algorithmes sont visibles dans la classe math.deca, ils portent le nom des fonctions trigonométriques (cos,sin...).

4.3 Sinus

Le premier algorithme de calcul à avoir été implémenté a été celui de la fonction sinus. Cette fonction peut être définie comme telle :

$$x \mapsto \sin(x)$$

$$\mathbb{R} \to [-1; 1]$$

4.3.1 Propriétés du sinus

Sinus est défini sur tout $\mathbb{R}$. Il faut alors utiliser ses proprietés pour le ramener dans un intervalle d'étude plus petit.
Proprietés sinus :

— Périodicité : $\forall k \in \mathbb{Z}$, $\sin(x + 2k\pi) = \sin(x)$

— Antisymétrie par rapport à 0 : $\sin(-x) = -\sin(x)$

— Symétrie par rapport à π : $\sin(\pi - x) = \sin(x)$

— Symétrie par rapport à $\pi/2$: $\sin(\pi/2 + x) = \sin(\pi/2 - x)$

— Relation entre cosinus et sinus pour $x \in [\pi/4; \pi/2]$: $\forall x \in [\pi/4; \pi/2], \sin(x) = \cos(\pi/2 - x)$

4.3.2 Choix des algorithmes

Ces différentes proprietés nous permettent alors de ramener notre étude d'intervalle à $[0; \pi/4]$. Etant donné cet intervalle les réels problématiques seraient ceux proche de 0. En effet la densité des flottants est très grande lorsqu'on se rapproche de 0. Un algorithme qui semble cohérent serait celui des séries de Taylor qui ont un terme d'erreur en $0x^n$.Or les valeurs proches de 0 ont alors une puissance à l'ordre n qui se rapproche fortement de 0.
Notre intervalle s'étend jusqu'à $[\pi/4]$, $\pi/4$ étant inférieur à 1, on pourrait songer à étendre notre algorithme reposant sur les séries de Taylor sur tout cet intervalle sans avoir à passer par d'autres algorithmes.

4.3.3 Développement en série de Taylor de Sinus

Développement en Série de taylor de sinus :

$$\sin(x) = \sum_{n=0}^{\infty}(-1)^n \frac{x^{2n+1}}{(2n+1)!}, +O(\frac{x^{2n+1}}{2n+1})$$

La série possède un rayon de convergence R=∞
On peut alors remarquer une suite inspirée de la série de Taylor ayant le lien suivant :

$$T_n = (-1)^n \frac{x^{2n+1}}{(2n+1)!}$$

$$T_{n+1} = (-1)^{n+1} \frac{x^{2(n+1)+1}}{(2(n+1)+1)!}$$

On a alors la relation :

$$T_{n+1} = T_n * \frac{(-1)}{(2n+2)(2n+3)}$$

L'algorithme repose alors sur ce lien. On initialise une somme avec le premier terme, ici x. Puis on itère en boucle en rajoutant à cette somme le terme suivant. Il y a deux options d'arrêt d'itération :
-soit la somme ne bouge plus ce qui signifie que le terme à rajouter n'est plus signifcatif
-soit on atteint un maximum d'itérations et on renvoie donc le résultat tel quel

4.3.4 Test de précision

Pour valider notre algorithme il faut alors étudier sur son domaine de définition la précision des résultats qu'il renvoie. Pour cela la fonction ulp(x) sera utilisée. Cette fonction est présentée dans la section autres fonctions mathématiques où son fonctionnement est décrit. Cette dernière renvoie la plus petite différence entre x et le flottant suivant. D'où le nom ULP(Unit in the Last Place). Cette fonction va être utile pour calculer l'erreur relative de notre implémentation. Nous allons aussi calculer l'erreur absolue de cette dernière en la comparant à la fonction sinus de la bibliothèque trigonométrique de java.

$$Erreurabsolue = |notresinus(x) - sinusjava(x)|$$

$$Erreurrelative = \frac{|notresinus(x) - sinusjava(x)|}{ulp(sinusjava(x))}$$

Pour avoir une bonne perception des erreurs de notre algorithme nous allons étudier ces erreurs sur plusieurs points du domaine.

Proche de 0 Ici les flottants sont denses et ont un ulp très petit.

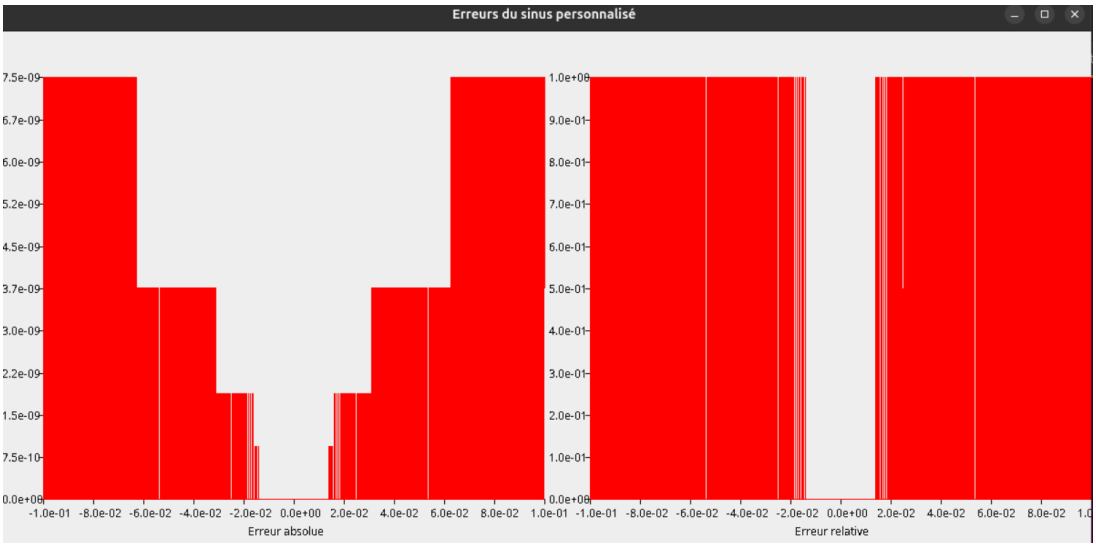


FIGURE 11 – Erreurs du sinus pour x proche de 0

On constate une très bonne précision de l'ordre de 10^{-9} ce qui correspond à l'ordre de grandeur du dernier bit car pour x proche de 0.1 on a un dernier bit a 2^{-24} soit de l'ordre de 10^{-8} ce qui est supérieur à notre précision. Lorsqu'on se rapproche de 0, cette erreur diminue encore fortement, pour l'analyse il est plus facile de regarder l'erreur relative. On observe une erreur relative d'un ulp proche de 0.1 et de 0 lorsqu'on se rapproche de 0 ce qui veut dire que notre valeur est exacte aux yeux des float.

Sur l'intervalle [-π ;π] : Domaine de définition de notre algorithme

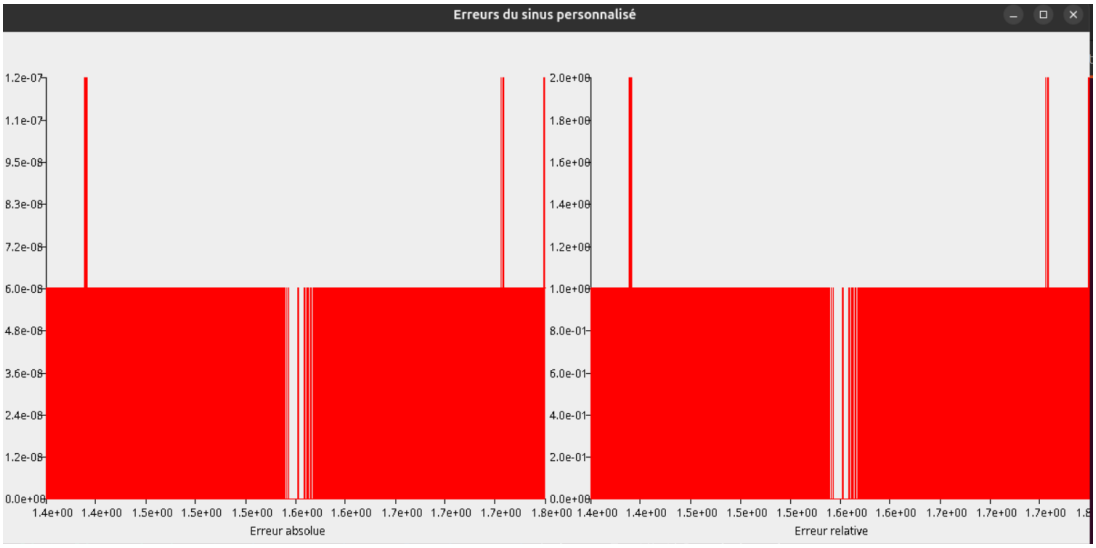


FIGURE 12 – Erreurs du sinus pour l'intervalle [-π ;π]

On constate une très bonne précision de l'ordre de 10^{-8} ce qui correspond à l'ordre de grandeur du dernier bit car pour x de l'ordre de 10^{0} on a un dernier bit à 2^{-23} soit de l'ordre de 10^{-7} ce qui est supérieur à notre précision.
On remarque encore une erreur relative de l'ordre de l'ulp ce qui correspond à nos attentes. Cependant on a ici exclu pi parce qu'autour de cette valeur sin(x) se rapproche de 0 et donc cela fait baisser l'ulp. On va alors étudier sur [-$2pi$;2π] pour observer ce problème.

Sur l'intervalle $[-2\pi; 2\pi]$ On veut étudier notre algorithme sur l'intervalle sans réduction modulaire pour voir si il es précis ici.

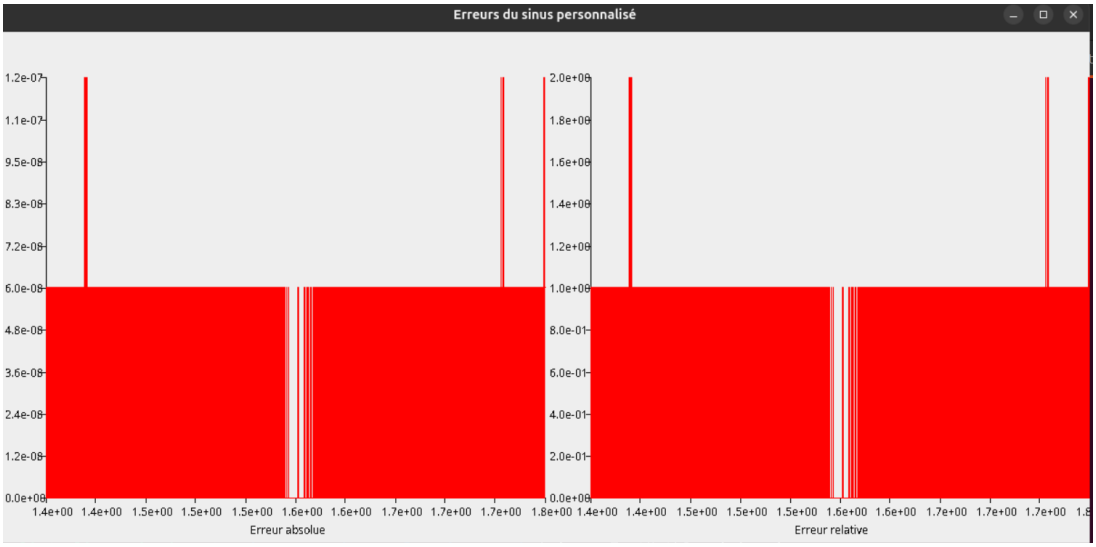


FIGURE 13 – Erreurs du sinus sur l'intervalle [-2π ;2π]

On remarque une chose bizarre, l'erreur absolue semble rester faible pour l'ensemble de l'intervalle mais proche des valeurs de -2pi et de 2pi on a une erreur relative très élevée. On peux expliquer cela par le fait que la valeur retournée par sinus(x) dans ces cas la se rapproche très fortement de 0 et donc la précision de l'ulp augmente énormément. On s'en rend bien compte ici lorsqu'on isole autour de π et de 2π :

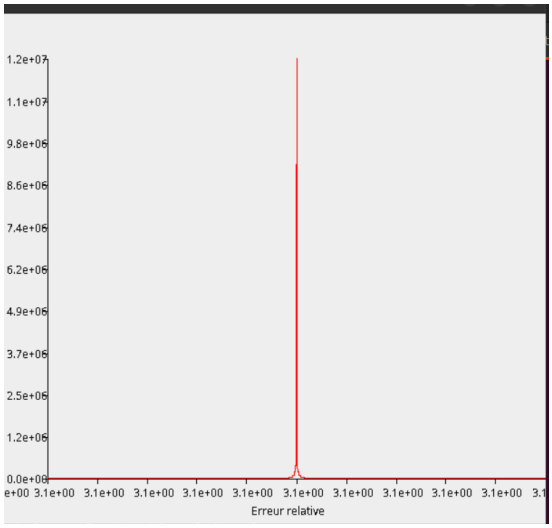


FIGURE 14 – Erreurs du sinus autour de pi

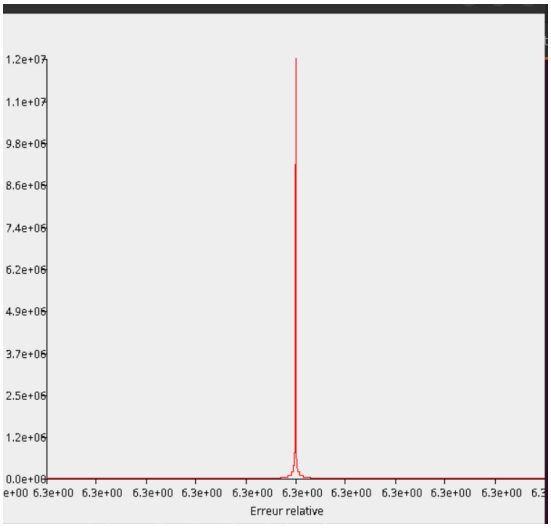


FIGURE 15 – Erreurs du sinus autour de 2pi

On peut isoler ces deux points et regarder les erreurs dans le reste de l'intervalle.

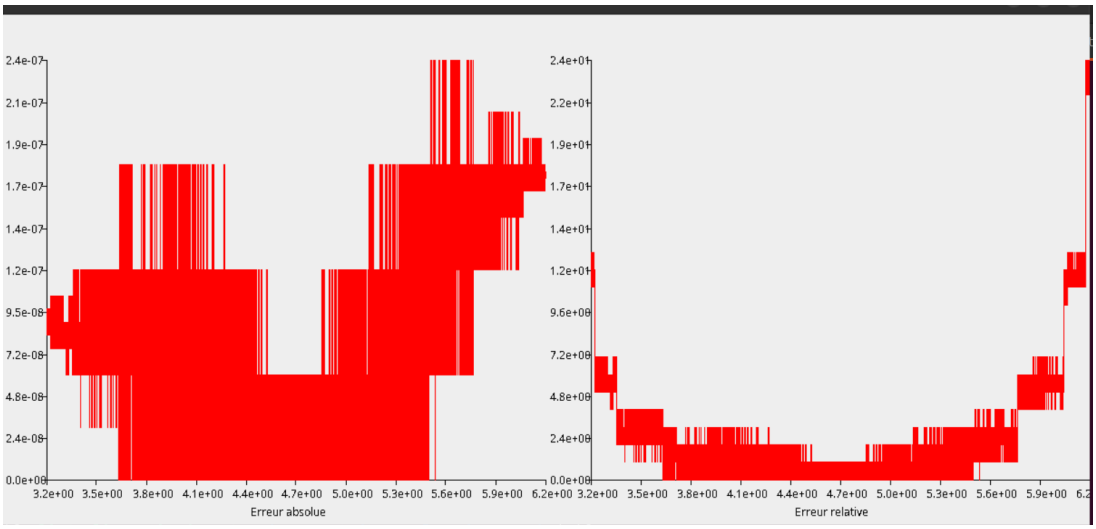


FIGURE 16 – Erreurs du sinus sur l'intervalle $]\pi\,;2\pi[$

Grands nombres On suppose ici une précision moindre du aux erreurs de réduction modulaire induites par nos algorithmes

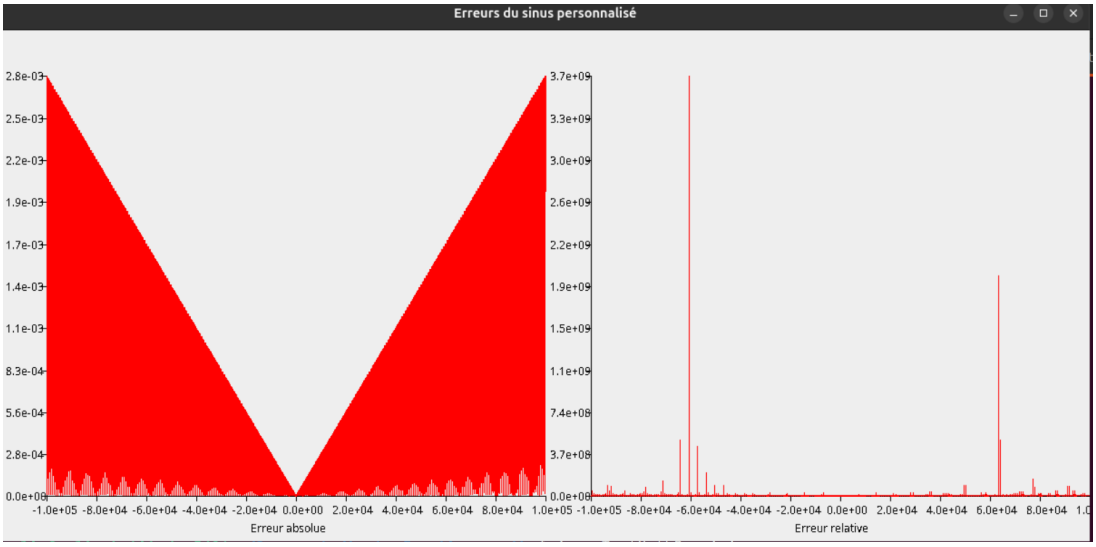


FIGURE 17 – Erreurs du sinus pour des grandes valeurs

On remarque ici que pour des grandes valeurs notre erreur absolue augmente linéairement avec le fait que nos valeurs soient grandes. On avait observé que des erreurs étaient introduites par notre réduction modulaire ce qui explique ce comportement. Mais on peut observer que la précision reste de l'ordre de 10^{-3} pour des valeurs de x de l'ordre de 10^5. Cependant les valeurs relatives en comparaison d'ulp sont mauvaises, le graphique est même inexploitable à cause d'erreurs très grandes induites par des réductions modulaires qui aurainet dues être proches de 0 et qui ne le sont pas.

Conclusion Est ce que notre critère de validation est validé ?
Notre algorithme rend une précision de sinus fiable sur l'ensemble de ses valeurs. On relève tout de même un manque de précision lorsque les valeurs de sortie sont proches de 0. Or pour contrecarer ce problème, notre algorithme analyse la valeur d'entrée et dans ce cas particuliers rend une valeur précise ex $\sin(2\pi)=0$. Cela permet tout de même de rester

fiable pour ces valeurs. Au niveau des grandes valeurs notre algorithme perd beaucoup, il serait possible de limiter sa plage d'actionn afin de garantir une précision. Par exemple pour garantir une précision de l'ordre de 10^{-4} il faudrait limiter nos valeurs a une valeur absolue de 10^{-4}.

4.3.5 Test de rapidité

Pour tester la rapidité de notre algorithme il a été décidé de le comparer avec la fonction de java en lui appliquant différents lots de valeurs de différents ordre de grandeur :

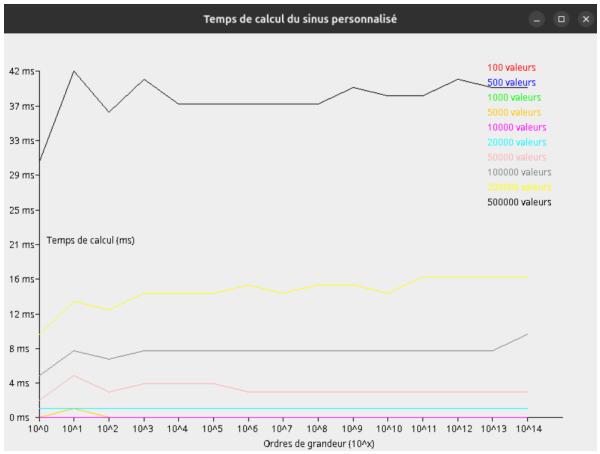


FIGURE 18 – Temps de calcul de notre algorithme

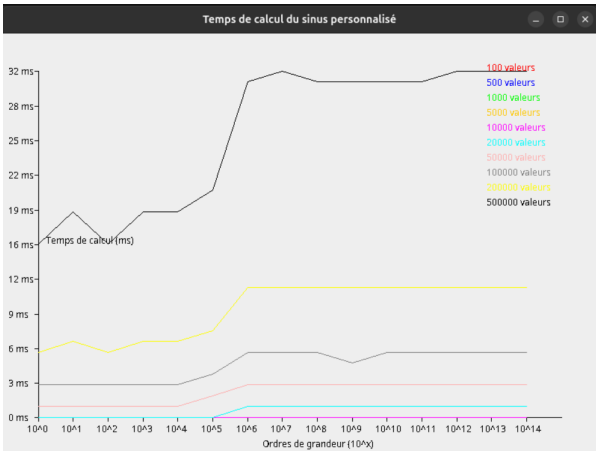


FIGURE 19 – Temps de calcul de sinus java

On peut alors valider notre algorithme qui possède les mêmes ordres de grandeurs que l'algorithme proposé par la bibliothèque math de java. Pour un lot de 500000 valeurs le temps d'execution n'est que d'environ 32ms.

4.4 Cosinus

Le second algoirithme implémenté est celui du cosinus étant donnée que celui du sinus en à besoin sur une certaine plage de valeurs. Cette dernière peut être définie comme telle :

$$x \mapsto \cos(x)$$
$$\mathbb{R} \to [-1; 1]$$

4.4.1 Propriétés du cosinus

Cosinus est défini sur tout $\mathbb{R}$. Il faut alors utiliser ses proprietés pour le ramener dans un intervalle d'étude plus petit.
Proprietés cosinus :

- Périodicité : $\forall k \in \mathbb{Z}$, $\cos(x + 2k\pi) = \cos(x)$
- Symétrie par rapport à 0 : $\cos(-x) = \cos(x)$
- Antisymétrie par rapport à π : $\cos(\pi - x) = -\cos(x)$
- Antisymétrie par rapport à $\pi/2$: $\cos(\pi/2 + x) = -\cos(\pi/2 - x)$
- Relation entre cosinus et sinus pour $x \in [\pi/4; \pi/2]$: $\forall x \in [\pi/4; \pi/2], \sin(x) = \cos(\pi/2 - x)$

4.4.2 Choix des algorithmes

Ces différentes proprietés nous permettent alors de ramener notre étude d'intervalle à $[0; \pi/4]$. On se retrouve alors dans le même cas que pour l'étude de notre sinus. Le choix du développement en série de taylor est envisagé.

4.4.3 Développement en série de Taylor de Cosinus

Développement en Série de taylor de cosinus :

$$\cos(x) = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n)!} + O(\frac{x^{2n}}{2n})$$

La série possède un rayon de convergence R=∞
On peut alors remarquer une suite inspirée de la série de Taylor ayant le lien suivant :

$$T_n = (-1)^n \frac{x^{2n}}{(2n)!}$$

$$T_{n+1} = (-1)^{n+1} \frac{x^{2(n+1)}}{(2(n+1))!}$$

On a alors la relation :

$$T_{n+1} = T_n * \frac{(-1)}{(2n+1)(2n+2)}$$

L'algorithme repose alors sur ce lien. On initialise une somme avec le premier terme, ici 1. Puis on itère en boucle en rajoutant à cette somme le terme suivant. Il y a deux options d'arrêt d'itération :
-soit la somme ne bouge plus ce qui signifie que le terme à rajouter n'est plus signifcatif
-soit on atteint un maximum d'itérations et on renvoie donc le résultat tel quel

4.4.4 Tests de précision

Pour valider notre algorithme de Cosinus on procède de la même manière que pour l'algorithme de sinus. On obtient les erreurs de la manière suivante :

$$Erreurabsolue = |notrecosinus(x) - cosinusjava(x)|$$

$$Erreurrelative = \frac{|notrecosinus(x) - cosinusjava(x)|}{ulp(cosinusjava(x))}$$

Proche de 0

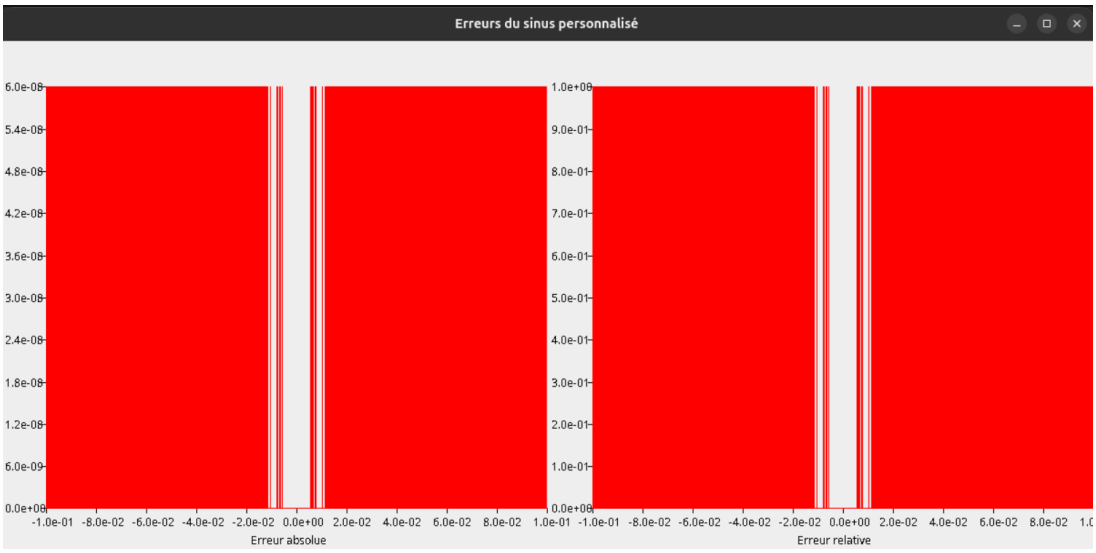


FIGURE 20 – Erreurs du cosinus pour x proche de 0

Sur l'intervalle [-$\pi/2$;$\pi/2$] :

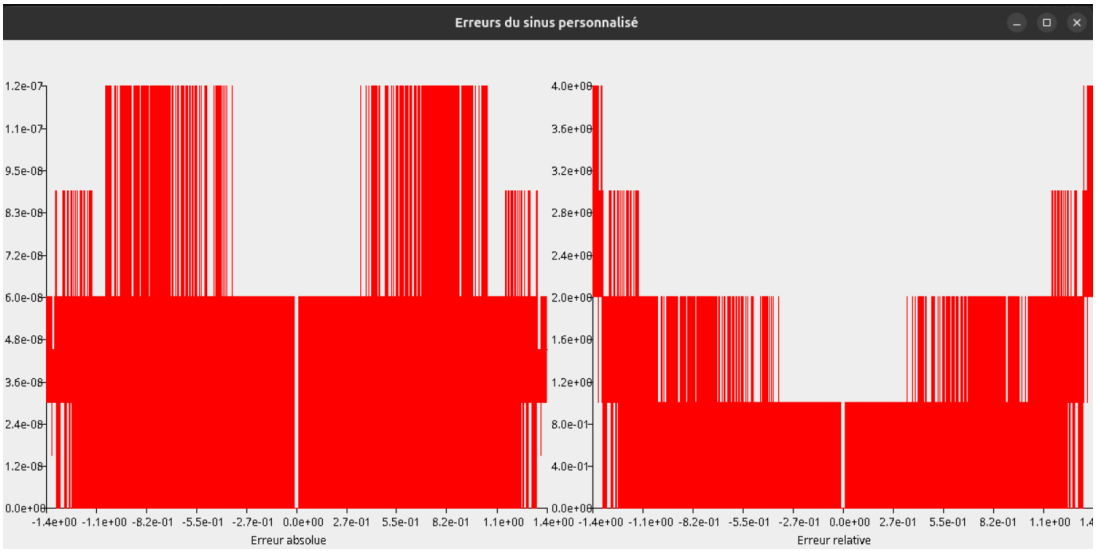


FIGURE 21 – Erreurs du cosinus pour x dans l'intervalle [-$\pi/2$;$\pi/2$]

Sur l'intervalle $[-2\pi; 2\pi]$ On veut étudier notre algorithme sur l'intervalle sans réduction modulaire pour voir si il es précis ici.

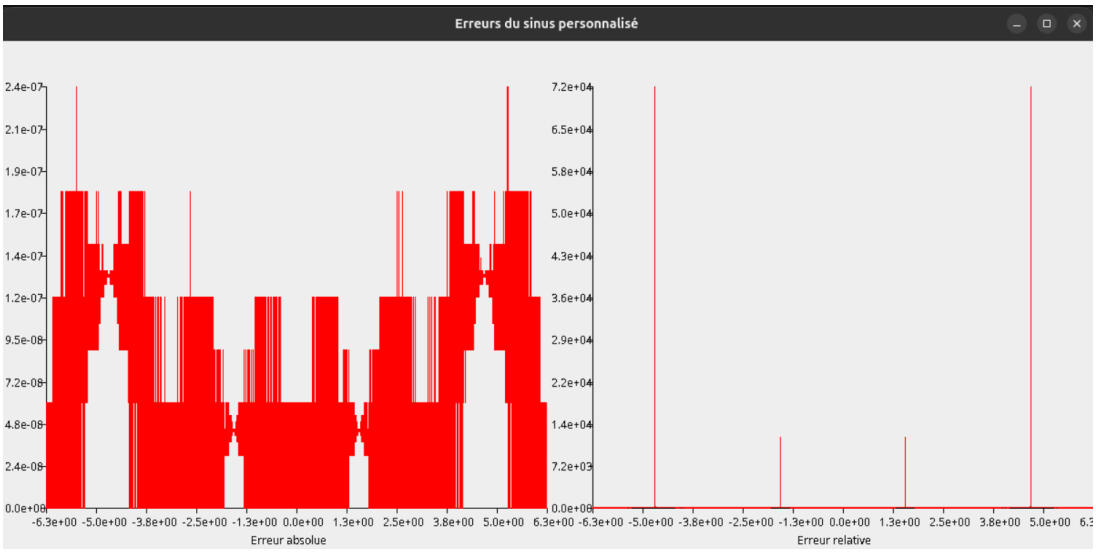


FIGURE 22 – Erreurs du cosinus sur l'intervalle [-2π ;2π]

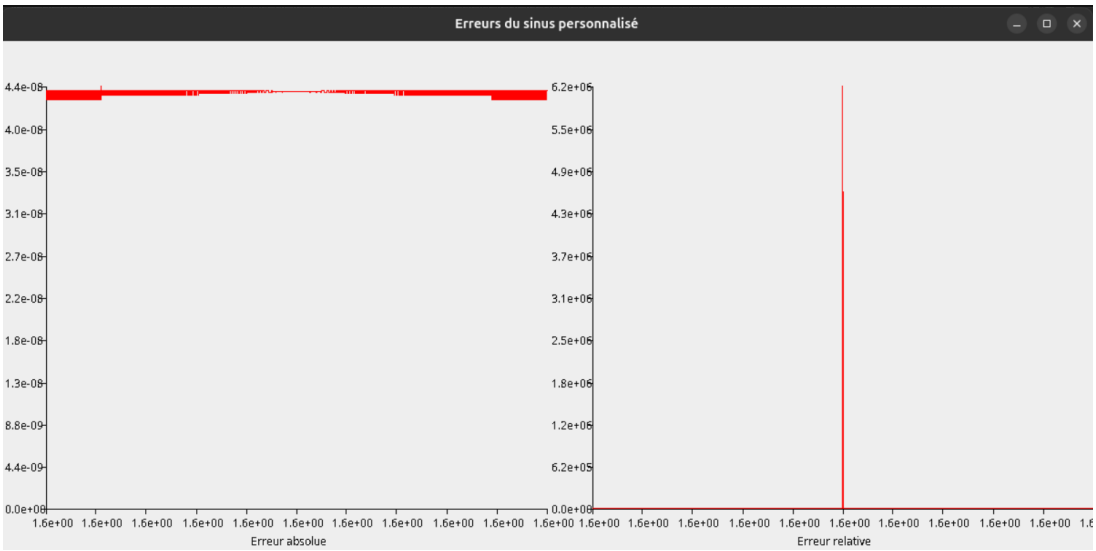


FIGURE 23 – Erreurs du cosinus autour de $\pi/2$

Grands nombres

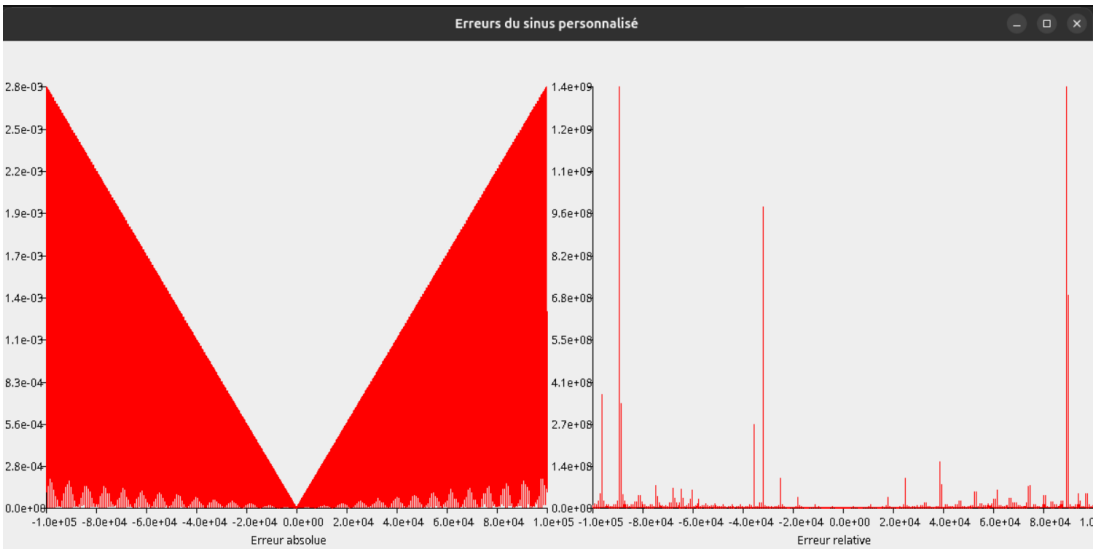


FIGURE 24 – Erreurs du cosinus pour des grandes valeurs

Conclusion Est ce que notre critère de validation est validé ?
Notre algorithme de cosinus présente les mêmes défauts et qualités que notre algorithme de sinus excepté que c'est pour des valeurs proche de $\pi/2$que les litiges apparaissent. Mais nous pouvons le valider de la même manière.

4.4.5 Test de rapidité

Pour tester la rapidité de notre algorithme il a été décidé de le comparer avec la fonction de java en lui appliquant différents lots de valeurs de différents ordre de grandeur :

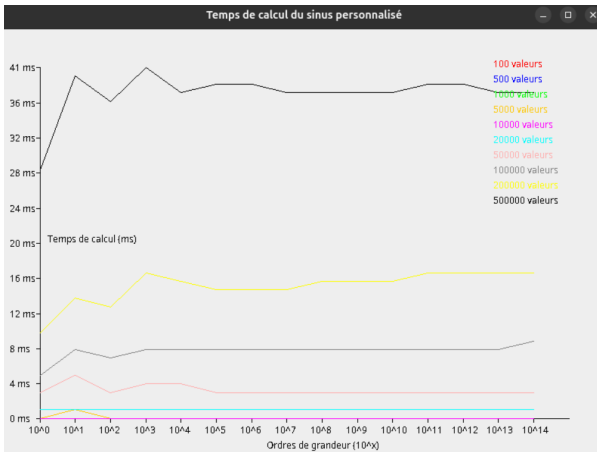


FIGURE 25 – Temps de calcul de notre algorithme

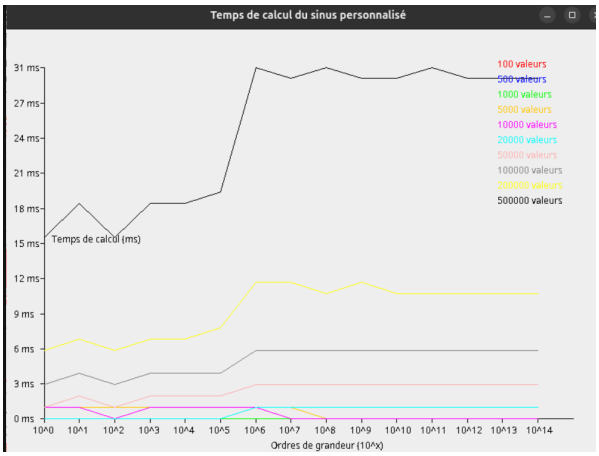


FIGURE 26 – Temps de calcul de cosinus java

On peut alors valider notre algorithme qui possède les mêmes ordres de grandeurs que l'algorithme proposé par la bibliothèque math de java. Pour un lot de 500000 valeurs le temps d'execution n'est que d'environ 31ms.

4.5 Tangente

La troisième fonction trigonométrique est la fonction tangente. Elle peut être définie comme telle :

$$x \mapsto \tan(x)$$
$$\{x \in \mathbb{R} | (x \neq \pi/2 + k\pi), k \in \mathbb{Z}\} \to \mathbb{R}$$

4.5.1 Propriétés de la tangente

Tangente est défini sur tout $\mathbb{R}$ hormis les points cités ci-dessus. Il faut alors utiliser ses proprietés pour le ramener dans un intervalle d'étude plus petit.
Proprietés tangente :

- Périodicité : $\forall k \in \mathbb{Z}$, $\tan(x + \pi) = \tan(x)$
- Antisymétrie par rapport à 0 : $\tan(-x) = -\tan(x)$
- Antisymétrie par rapport à $\pi/2$: $\tan(x) = -\tan(\pi - x)$
- Défintion de tangente : $\tan x = \frac{\sin x}{\cos x}$

4.5.2 Choix des algorithmes

On peut alors réduire notre intervalle d'étude à $[0; \pi/2]$. Sauf que la dernière définition peut permettre de déduire le calcul de tangente à l'aide de nos algorithmes de calcul de cosinus et sinus si ces derniers sont suffisemment précis et rapides pour permettre un calcul efficace de tangente. Ce choix est accentué du fait que le dévloppement en série de Taylor de tangente fait intervenir des termes plus compliqués que ceux du cosinus et sinus.

4.5.3 Tests de Précision

Pour valider notre algorithme de Tangente on procède de la même manière que pour l'algorithme de sinus et cosinus. On obtient les erreurs de la manière suivante :

$$Erreurabsolue = |notretangente(x) - tangentejava(x)|$$
$$Erreurrelative = \frac{|notretangente(x) - tangentejava(x)|}{ulp(tangentejava(x))}$$

Proche de 0 Notre fonction tangente est censée renvoyer 0 proche de 0, ce qui peut créer des problèmes de précision :

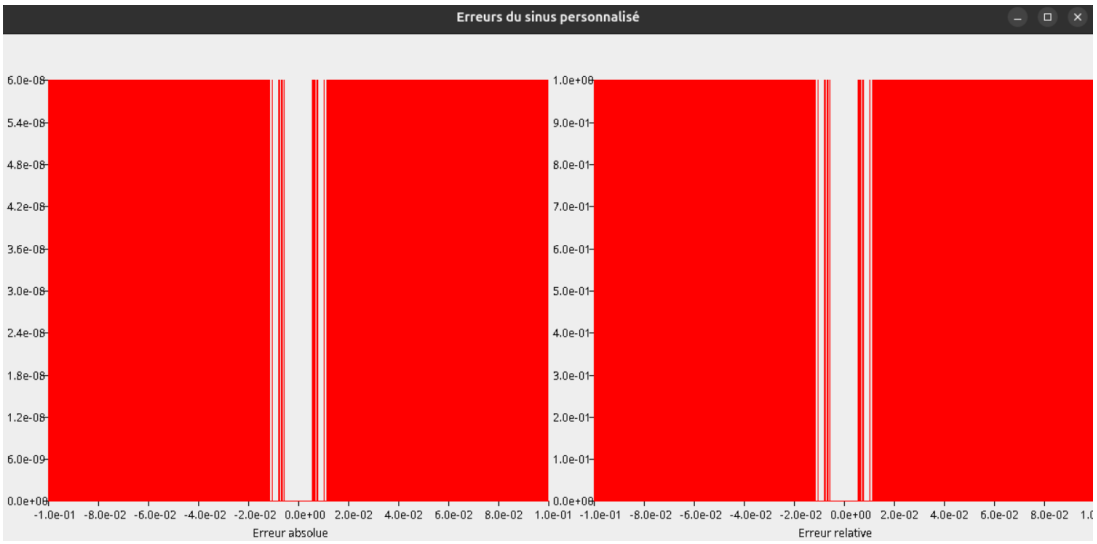


FIGURE 27 – Erreurs de la tangente pour x proche de 0

On observe alors une erreur absolue faible autour de 0 et une précision de l'ordre de l'ulp pour les valeurs dans cet intervalle ce qui nous permet de dire que le comportement de notre algorithme est ici celui attendu.

Proche de $\pi/2$ La tangente possède des discontinuités pour les x proche de $\pi/2 + k\pi$. Notre algorithme renvoyant une erreur cela pourrait poser un soucis.

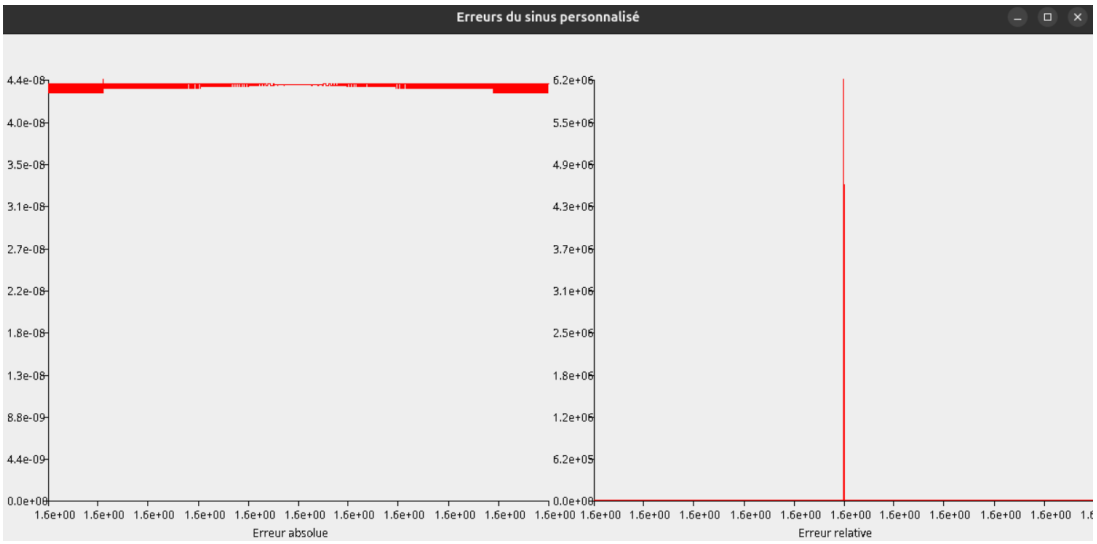


FIGURE 28 – Erreurs de la tangente pour x proche de $\pi/2$

On observe ici qu'il y a un problème important de résultat mais quand on regarde précisemment, pour le cas $\tan(\pi/2)$ la bibliothèque mathématique java renvoie un float=INFINI. Or les float tels que NaN, INFINITE et autre ne sont pas implémentables en DECA donc le choix de ne rien renvoyer est fait par notre algorithme ce qui pose un soucis dans les différences en $\pi/2$ pour l'erreur relative. Sinon, l'erreur absolue autour de ce point est très faible. Nous pourvons alors peut être regarder en enlevant le point $\pi/2$.

Sur l'intervalle $[\pi; \pi]$ On veut étudier notre algorithme sur son intervalle sans appel à la réduction modulaire (on exclut ici le cas $x = \pi/2$).

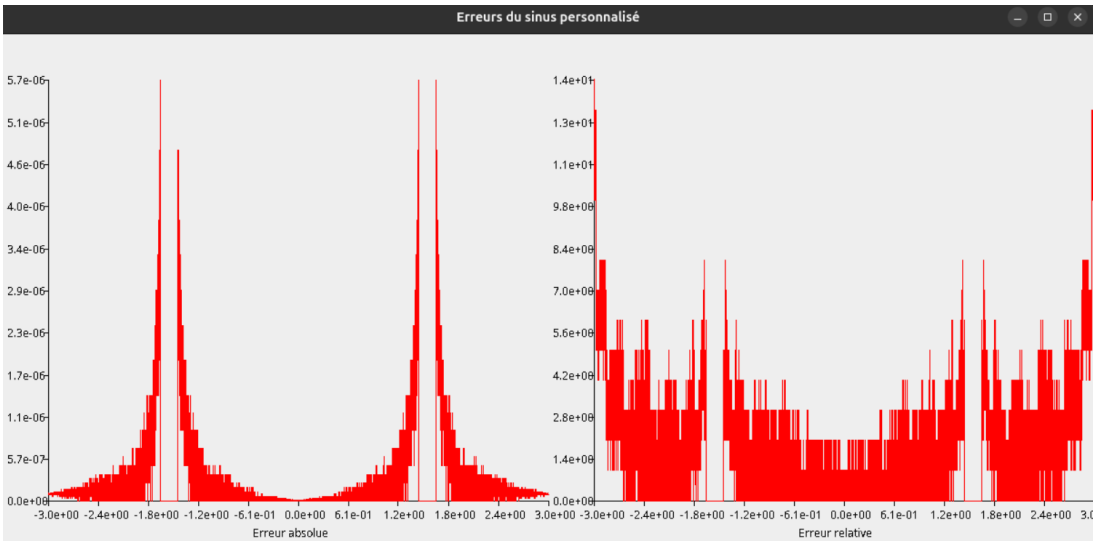


FIGURE 29 – Erreurs du cosinus sur l'intervalle [-2π ;2π]

On observe ici une divergence des valeurs proche de nos valeurs remarquables de $\pi/2$ mais sinon une erreur relative de l'ordre de quelques ulp pour la plupart des calculs.

L'étude sur R retournerait un résultat incohérent et biaisé par les erreurs sur toutes les valeurs de type $\pi/2 + k\pi$. On se doute du même type de résultats que sur les fonctions cosinus et sinus où l'erreur principale vient de la réduction modulaire.

4.5.4 Test de rapidité

Pour tester la rapidité de notre algorithme il a été décidé de le comparer avec la fonction de java en lui appliquant différents lots de valeurs de différents ordre de grandeur :

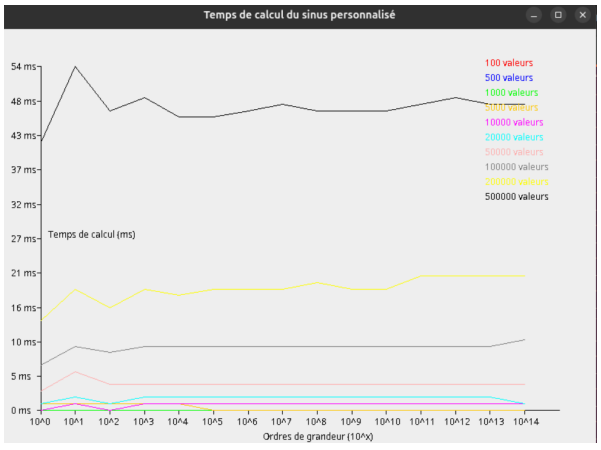


FIGURE 30 – Temps de calcul de notre algorithme

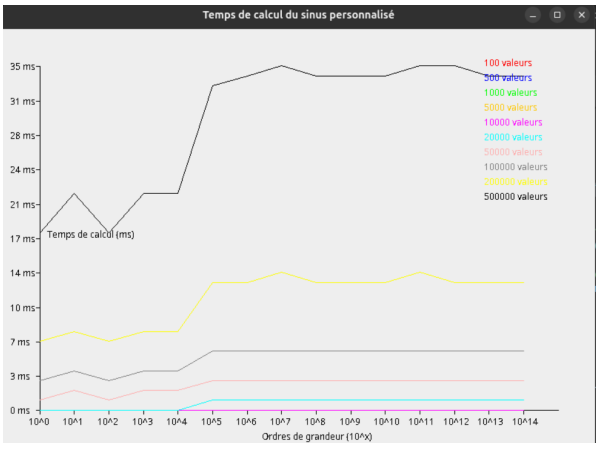


FIGURE 31 – Temps de calcul de tangente java

On peut alors tout de même valider la rapidité de notre algorithme qui à cause de son choix d'implémentation prends 2 fois plus de temps que celui de java du à son double appel à sinus et à cosinus. Mais pour calculer 5000 000 valeurs ce dernier met environ 50ms ce qui reste rapide.

4.6 Arcsinus

Cette fonction peut être définie comme telle :

$$x \mapsto \arcsin(x)$$

$$[-1;1] \to [-\pi/2;\pi/2]$$

4.6.1 Propriétés de l'arcsinus

L'arcsinus est défini uniquement sur l'intervalle $[-1;1]$, ce qui limite notre domaine d'étude. Il est néanmoins nécessaire d'exploiter ses propriétés pour simplifier le calcul et optimiser sa précision.
Propriétés de l'arcsinus :

- Antisymétrie par rapport à 0 : $\arcsin(-x) = -\arcsin(x)$
- Complémentarité dans $[0;1]$: Pour $x > 0.5$, il est avantageux d'utiliser la relation :

$$\arcsin(x) = \frac{\pi}{2} - 2\arcsin\left(\sqrt{\frac{1-x}{2}}\right)$$

  Cette propriété permet de ramener les calculs à une plage plus restreinte où les approximations convergent rapidement.

4.6.2 Choix des algorithmes

En utilisant les propriétés de l'arcsinus, il est possible de ramener notre domaine d'étude à $[0;0.5]$. Sur cet intervalle, la fonction est relativement simple et ne nécessite pas de transformations complexes. Cependant, il est nécessaire d'adopter des stratégies adaptées pour traiter les cas limites proches de 0 ou de 0.5.

Pour $[0;0.5]$, un algorithme basé sur les séries de Taylor est particulièrement adapté. Les séries de Taylor convergent rapidement pour des valeurs proches de 0, car le terme d'erreur diminue en proportion de x^n. Pour $x > 0.5$, nous exploitons la relation complémentaire décrite ci-dessus pour ramener le problème dans $[0;0.5]$, garantissant ainsi une précision accrue et une convergence rapide.

4.6.3 Développement en série de Taylor de l'arcsinus

Le développement limité de $\arcsin(x)$ autour de $x = 0$ est donné par la série suivante :

$$\arcsin(x) = \sum_{n=0}^{\infty} \frac{(2n)!}{2^{2n}(n!)^2(2n+1)} x^{2n+1} + \mathcal{O}(x^{2n+3}).$$

La série possède un rayon de convergence $R = 1$, ce qui correspond exactement à l'intervalle de définition de la fonction arcsinus $[-1;1]$.

Nous définissons les termes successifs de cette série comme suit :

$$T_n = \frac{(2n)!}{2^{2n}(n!)^2(2n+1)} x^{2n+1},$$

$$T_{n+1} = T_n \cdot \frac{(2n+1)^2 * (2n+2)}{4(n+1)(2n+3)} \cdot x^2.$$

L'algorithme repose sur cette relation de récurrence. L'implémentation suit les étapes suivantes :

— On initialise la somme avec le premier terme $T_0 = x$.

— À chaque itération, on calcule T_{n+1} en utilisant la relation de récurrence et on l'ajoute à la somme.

— Le processus s'arrête soit lorsque le terme ajouté devient négligeable, soit lorsqu'un nombre maximal d'itérations est atteint.

4.6.4 Tests de Précision

Pour valider notre algorithme d'arcsinus, nous procédons de manière similaire aux tests pour sinus et cosinus. Les erreurs sont définies comme suit :

$$\text{Erreur absolue} = |\text{notrearcsin}(x) - \arcsin_{\text{java}}(x)|$$

$$\text{Erreur relative} = \frac{|\text{notrearcsin}(x) - \arcsin_{\text{java}}(x)|}{\text{ulp}(\arcsin_{\text{java}}(x))}$$

Proche de 0 Notre fonction arcsinus est censée renvoyer des résultats proches de x lorsque x est proche de 0. Cet intervalle est particulièrement sensible car la densité des flottants est élevée autour de 0, ce qui peut affecter la précision :

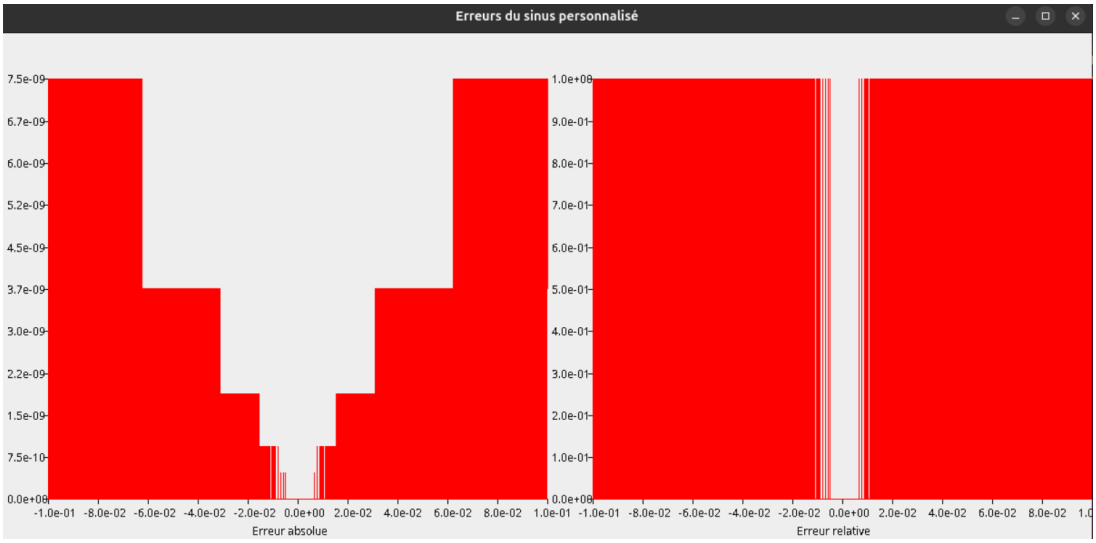


FIGURE 32 – Erreurs de l'arcsinus pour x proche de 0

On observe ici une erreur absolue très faible autour de 0. L'erreur relative est également de l'ordre de l'ulp, ce qui montre que l'algorithme conserve une bonne précision dans cet intervalle.

Sur l'intervalle $[-1; 1]$ Nous étudions ici la précision de l'algorithme d'arcsinus sur son domaine de définition complet $[-1; 1]$. Ce test exclut explicitement les points problématiques liés à des valeurs hors domaine ou des erreurs numériques extrêmes.

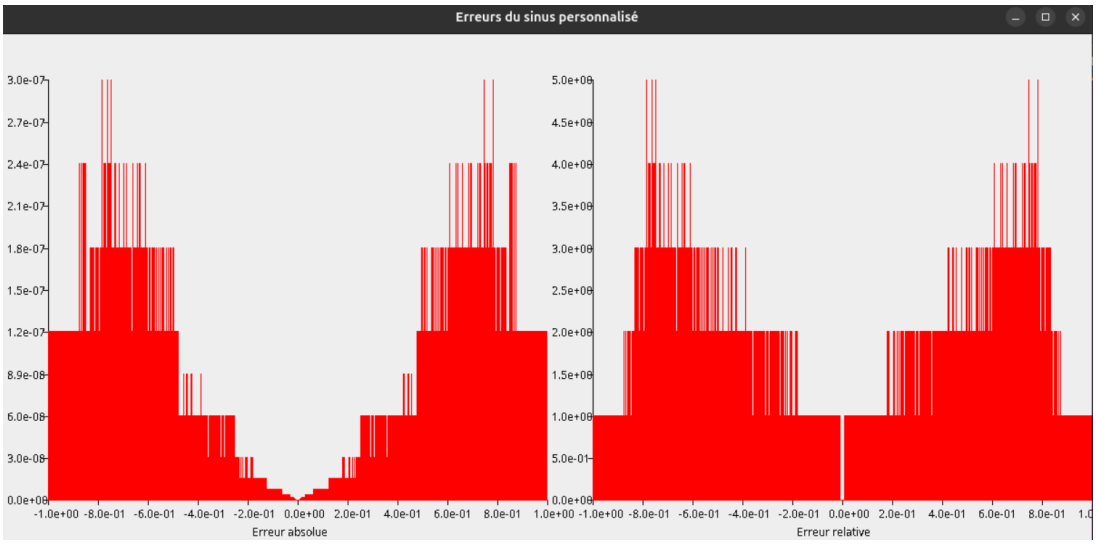


FIGURE 33 – Erreurs de l'arcsinus sur l'intervalle $[-1; 1]$

Sur l'ensemble de l'intervalle, on observe une erreur relative de l'ordre de quelques ulp pour la majorité des calculs. Ce qui permet de valider la précision de notre arcsinus sur l'ensemble de cet intervalle.

Analyse La précision de notre algorithme Arcsinus est donc très bonne sur l'ensemble de son ensemble de défintion, elle est de 5 ulp maximum ce qui nous permet de valider ce critère.

4.6.5 Tests de Rapidité

Le test de rapidité se fait en observant le temps de calcul dans l'intervalle [-1 ;1] de plusieurs jeux de valeurs de taille variée.

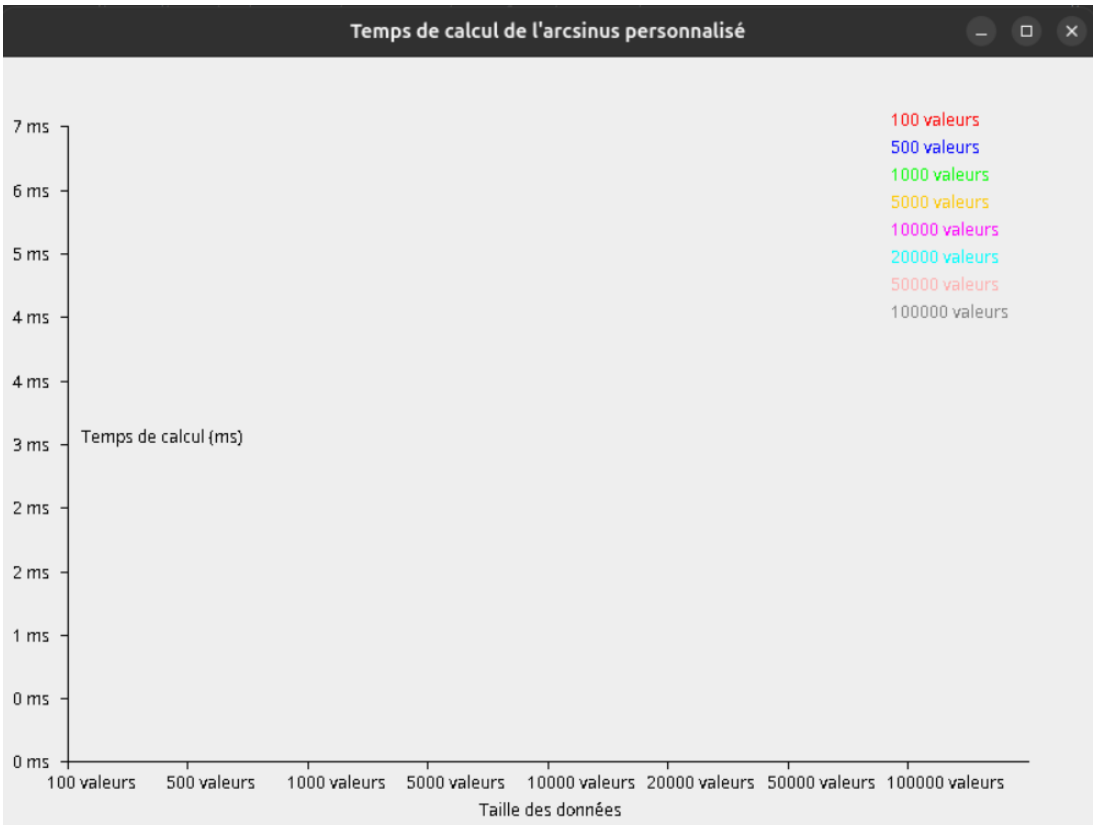


FIGURE 34 – Temps de calcul de arcsinus

Comme on peut le voir le fait de ne pas avoir à effectuer de réduction modulaire rend le calcul très rapide et inférieur à la milliseconde même pour 100000 données. On valide donc la rapidité de notre algorithme.

4.7 Arccosinus

Cette fonction peut être définie comme telle :

$$x \mapsto \arccos(x)$$

$$[-1;1] \to [0;\pi]$$

4.7.1 Propriétés de l'arccosinus

L'arccosinus est défini uniquement sur l'intervalle $[-1;1]$, ce qui limite notre domaine d'étude. Cette fonction possède une relation directe avec l'arcsinus, que nous exploitons pour simplifier son calcul.

Propriétés de l'arccosinus :

— Relation avec l'arcsinus : Pour $x \in [-1;1]$,

$$\arccos(x) = \frac{\pi}{2} - \arcsin(x).$$

Cette relation permet de réduire le calcul de l'arccosinus à celui de l'arcsinus, qui a déjà été optimisé.

— Symétrie pour $x \in [-1; 1]$:

$$\arccos(-x) = \pi - \arccos(x).$$

Cette propriété est utile pour étendre le calcul à tout l'intervalle.

4.7.2 Choix des algorithmes

Pour calculer l'arccosinus, nous utilisons principalement la relation :

$$\arccos(x) = \frac{\pi}{2} - \arcsin(x).$$

Cette approche garantit une précision identique à celle de l'arcsinus, tout en simplifiant considérablement l'implémentation.

En cas de besoin, la symétrie de l'arccosinus peut être utilisée pour ramener les calculs dans une plage restreinte $[0; 1]$, en appliquant $\arccos(-x) = \pi - \arccos(x)$.

4.7.3 Tests de Précision

Pour valider notre algorithme d'arccosinus, nous procédons de manière similaire aux tests pour arcsinus. Les erreurs sont définies comme suit :

$$\text{Erreur absolue} = |\text{notrearccos}(x) - \arccos_{\text{java}}(x)|$$

$$\text{Erreur relative} = \frac{|\text{notrearccos}(x) - \arccos_{\text{java}}(x)|}{\text{ulp}(\arccos_{\text{java}}(x))}$$

Sur l'intervalle $[0; 1]$ Notre fonction arccosinus est censée renvoyer des résultats proches de 0 lorsque $x \to 1$. Cet intervalle est particulièrement sensible car la densité des flottants est élevée autour de 1, ce qui peut affecter la précision :

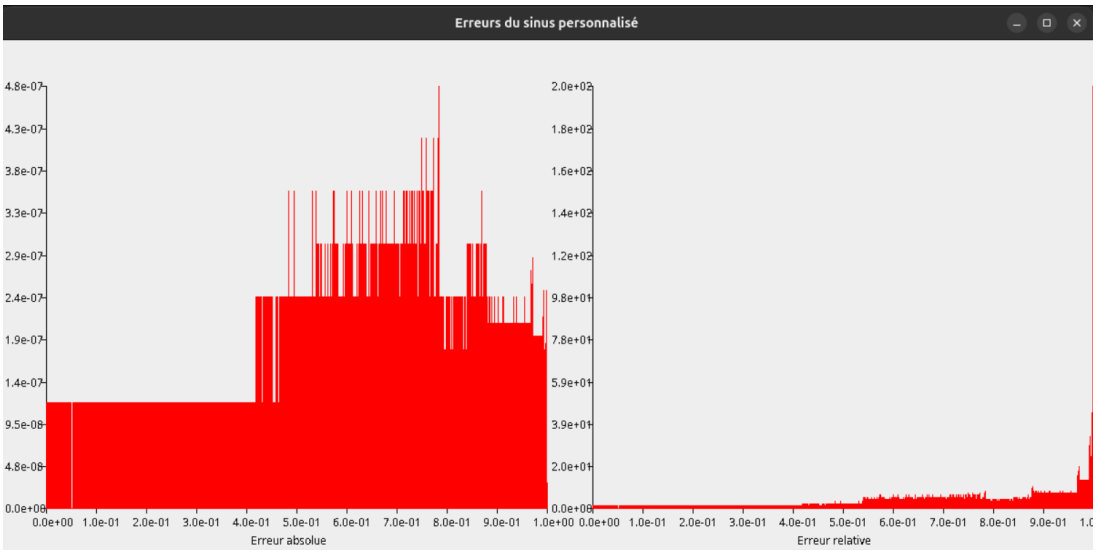


FIGURE 35 – Erreurs de l'arccosinus sur l'intervalle $0; 1]$

On observe ici une erreur absolue très faible sur l'intervalle mais la densité des flottants fait que l'erreur relative est très grande quand on s'approche de 1. Et cela pour une raison inexpliquée, on obtient des erreurs de l'ordre de 20ulp ce qui invalide notre test de précision sur cette plage de valeurs.

Sur l'intervalle $[-1; 0]$ Nous étudions ici la précision de l'algorithme d'arccosinus sur $[0; 1]$.

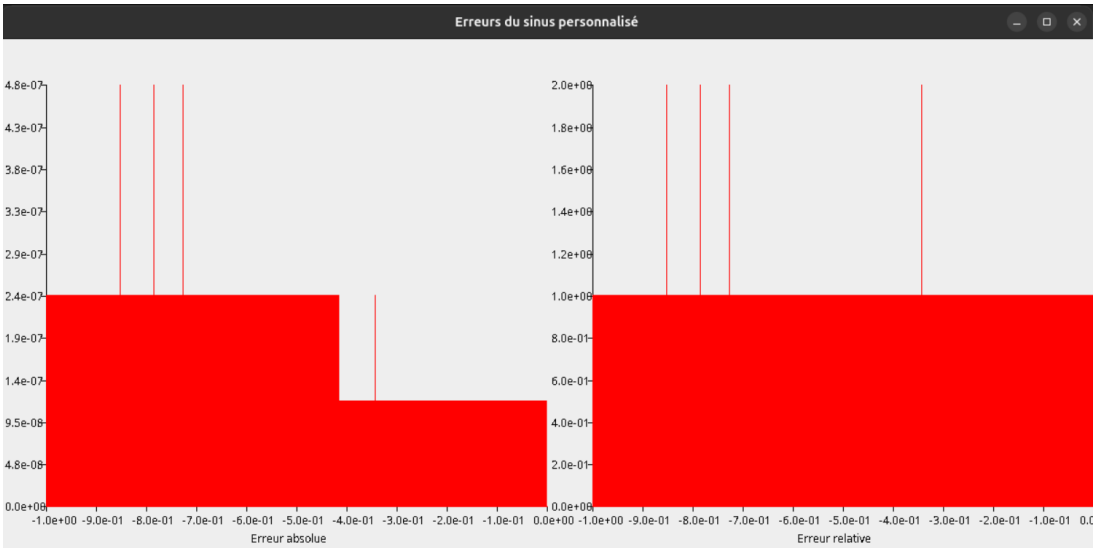


FIGURE 36 – Erreurs de l'arccosinus sur l'intervalle $[-1; 1]$

Sur l'ensemble de l'intervalle, on observe une erreur relative de l'ordre de quelques ulp pour la majorité des calculs. Ce qui permet de valider la précision de notre arccosinus sur l'ensemble de cet intervalle. La précision de notre algorithme Arccosinus est directement liée à celle de l'arcsinus, mais pour des raisons non élucidées les résultats proche de x=1 pour arccos sont trop peu précis (même si le nombre maximum d'itérations n'est pas atteint) pour valider le test de notre algorithme arccos.

4.7.4 Tests de Rapidité

Le test de rapidité se fait en observant le temps de calcul dans l'intervalle $[-1; 1]$ de plusieurs jeux de valeurs de taille variée.

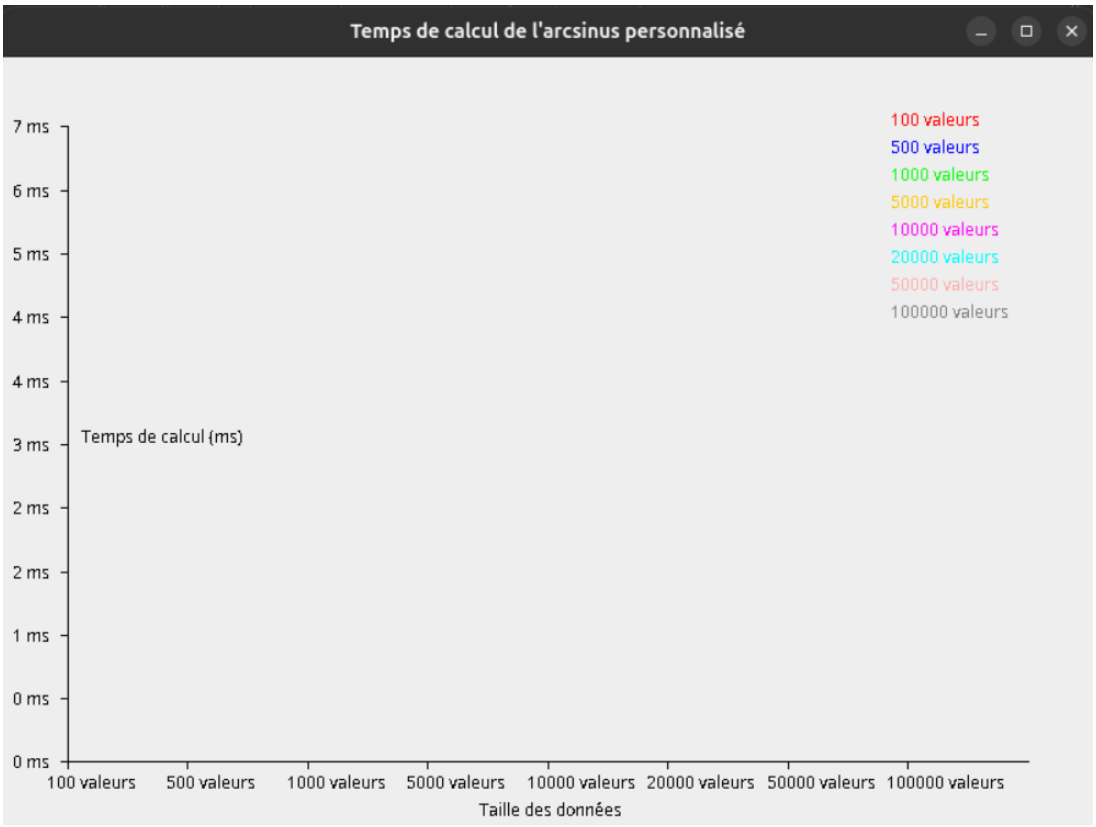


FIGURE 37 – Temps de calcul de l'arccosinus

Comme on peut le voir, l'utilisation de la relation $\arccos(x) = \frac{\pi}{2} - \arcsin(x)$ permet un calcul très rapide. Le temps de calcul reste inférieur à la milliseconde même pour 100000 données. Nous validons donc la rapidité de notre algorithme.

4.8 Arccosinus

Cette fonction peut être définie comme telle :

$$x \mapsto \arccos(x)$$

$$[-1; 1] \to [0; \pi]$$

4.8.1 Propriétés de l'arccosinus

L'arccosinus est défini uniquement sur l'intervalle $[-1; 1]$, ce qui limite notre domaine d'étude. Cette fonction possède une relation directe avec l'arcsinus, que nous exploitons pour simplifier son calcul.

Propriétés de l'arccosinus :

— Relation avec l'arcsinus : Pour $x \in [-1; 1]$,

$$\arccos(x) = \frac{\pi}{2} - \arcsin(x).$$

Cette relation permet de réduire le calcul de l'arccosinus à celui de l'arcsinus, qui a déjà été optimisé.

— Symétrie pour $x \in [-1; 1]$:

$$\arccos(-x) = \pi - \arccos(x).$$

Cette propriété est utile pour étendre le calcul à tout l'intervalle.

4.8.2 Choix des algorithmes

Pour calculer l'arccosinus, nous utilisons principalement la relation :

$$\arccos(x) = \frac{\pi}{2} - \arcsin(x).$$

Cette approche garantit une précision identique à celle de l'arcsinus, tout en simplifiant considérablement l'implémentation.

En cas de besoin, la symétrie de l'arccosinus peut être utilisée pour ramener les calculs dans une plage restreinte $[0; 1]$, en appliquant $\arccos(-x) = \pi - \arccos(x)$.

4.8.3 Tests de Précision

Pour valider notre algorithme d'arccosinus, nous procédons de manière similaire aux tests pour arcsinus. Les erreurs sont définies comme suit :

$$\text{Erreur absolue} = |\text{notrearccos}(x) - \arccos_{\text{java}}(x)|$$

$$\text{Erreur relative} = \frac{|\text{notrearccos}(x) - \arccos_{\text{java}}(x)|}{\text{ulp}(\arccos_{\text{java}}(x))}$$

Proche de 1 Notre fonction arccosinus est censée renvoyer des résultats proches de 0 lorsque $x \to 1$. Cet intervalle est particulièrement sensible car la densité des flottants est élevée autour de 1, ce qui peut affecter la précision :

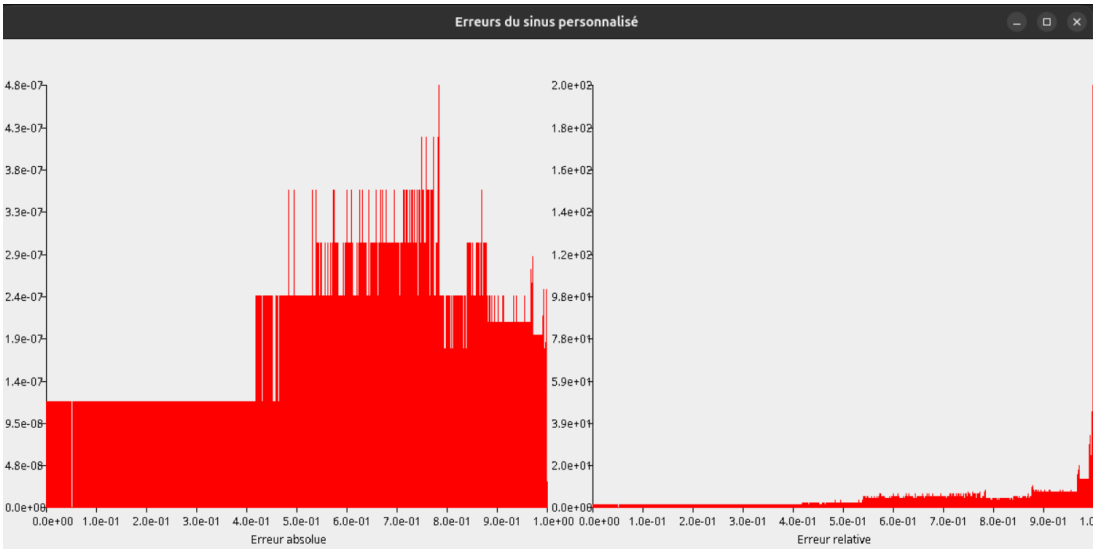


FIGURE 38 – Erreurs de l'arccosinus pour x proche de 1

On observe ici une erreur absolue très faible autour de 1 mais significative au vu de la densité des flottants. Cette erreur est donc trop élevée au vu de l'exigence d'erreur relative.

Sur l'intervalle $[-1; 0]$ Nous étudions ici la précision de l'algorithme d'arccosinus sur son domaine de définition inférieur à 0 pour ne pas avoir les erreur dues à arccos(1) $[-1; 0]$. Ce test exclut explicitement les points problématiques liés à des valeurs hors domaine ou des erreurs numériques extrêmes.

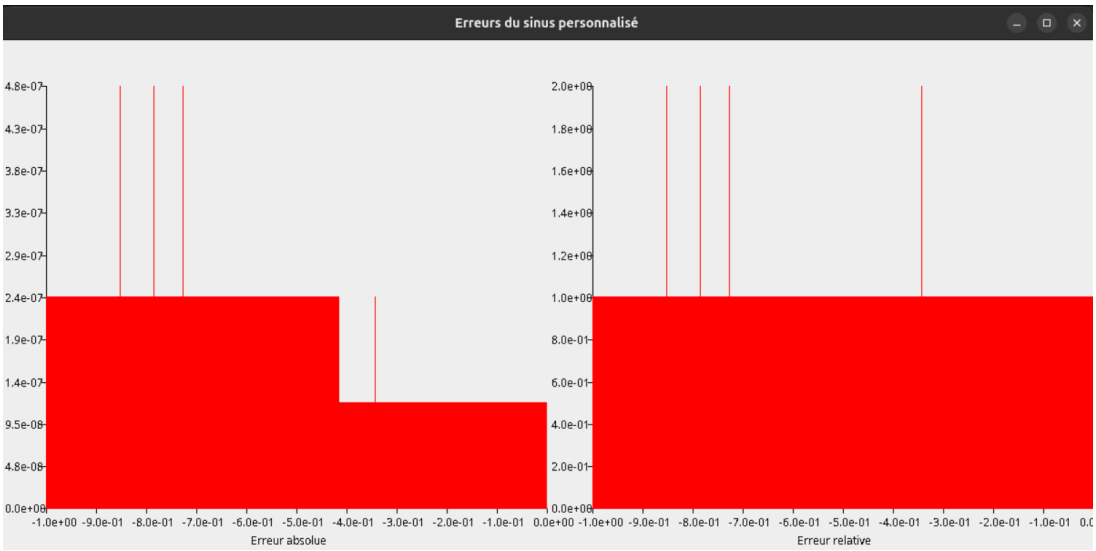


FIGURE 39 – Erreurs de l'arccosinus sur l'intervalle $[-1; 1]$

Sur l'ensemble de l'intervalle, on observe une erreur relative de l'ordre de quelques ulp pour la majorité des calculs. Ce qui permet de valider la précision de notre arccosinus sur l'ensemble de cet intervalle.

4.8.4 Tests de Rapidité

Le test de rapidité se fait en observant le temps de calcul dans l'intervalle $[-1; 1]$ de plusieurs jeux de valeurs de taille variée.

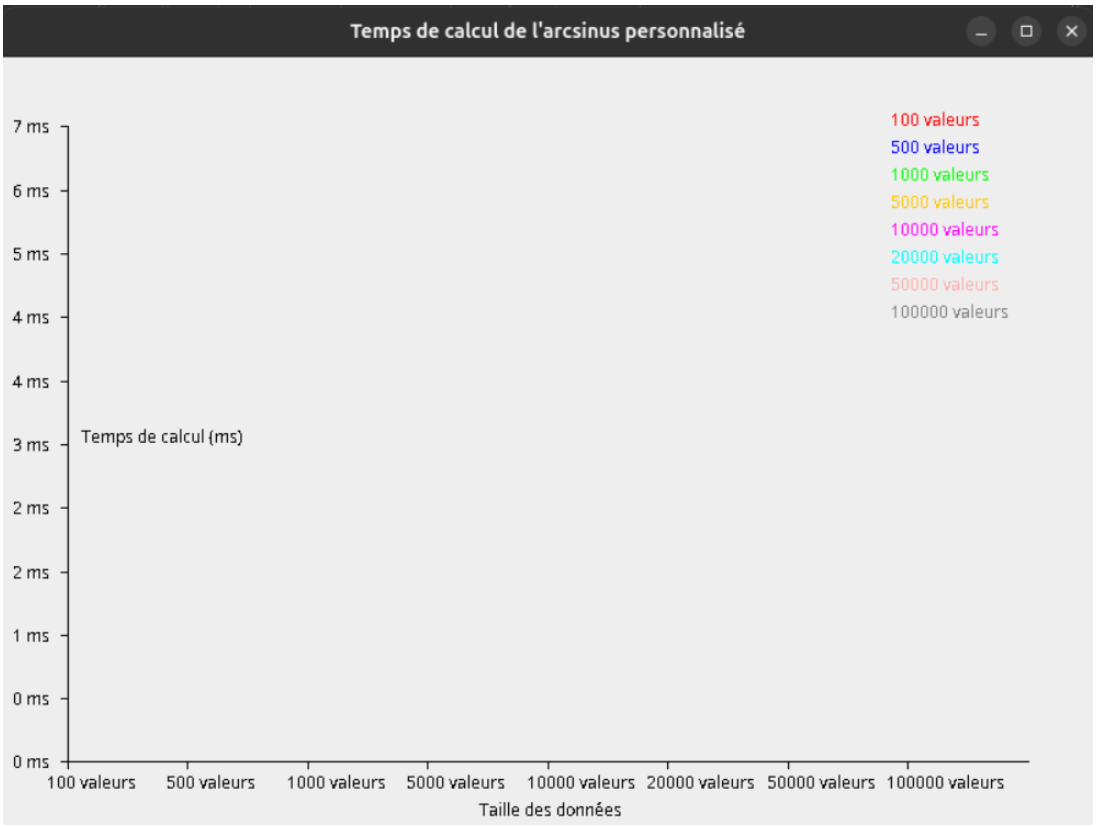


FIGURE 40 – Temps de calcul de l'arccosinus

Comme on peut le voir, l'utilisation de la relation $\arccos(x) = \frac{\pi}{2} - \arcsin(x)$ permet un calcul très rapide. Le temps de calcul reste inférieur à la milliseconde même pour 10000 données. Nous validons donc la rapidité de notre algorithme.

4.8.5 Analyse des tests

La précision de notre algorithme Arccosinus est directement liée à celle de l'arcsinus, avec des erreurs absolues faibles mais des erreurs relatives maximales très grandes inexpliquées pour arccos(1). Ce test invalide donc la précision requise pour arccos sur son ensemble de définition.

4.9 Arctangente

Cette fonction peut être définie comme telle :

$$x \mapsto \arctan(x)$$

$$\mathbb{R} \to \left[-\frac{\pi}{2}; \frac{\pi}{2}\right]$$

4.9.1 Propriétés de l'arctangente

L'arctangente est définie sur tout $\mathbb{R}$. Cependant, pour garantir une précision optimale, il est utile d'exploiter ses propriétés pour simplifier le calcul et améliorer sa convergence. Propriétés de l'arctangente :

— Antisymétrie par rapport à 0 : $\arctan(-x) = -\arctan(x)$

— Relation complémentaire pour $x > 1$:

$$\arctan(x) = \frac{\pi}{2} - \arctan\left(\frac{1}{x}\right)$$

Cette propriété permet de ramener les calculs dans l'intervalle $[0, 1]$, où les approximations convergent rapidement.

— Comportement asymptotique :

$$\lim_{x \to +\infty} \arctan(x) = \frac{\pi}{2}, \quad \lim_{x \to -\infty} \arctan(x) = -\frac{\pi}{2}.$$

4.9.2 Choix des algorithmes

En exploitant les propriétés de l'arctangente, il est possible de ramener le calcul dans l'intervalle $[0; 1]$, où les séries de Taylor convergent rapidement.

Pour $x \in [0; 1]$, un algorithme basé sur les séries de Taylor est particulièrement adapté. Les séries de Taylor convergent efficacement pour des valeurs proches de 0, car l'erreur diminue proportionnellement à x^{2n+1}.

Pour $x > 1$, nous exploitons la relation complémentaire :

$$\arctan(x) = \frac{\pi}{2} - \arctan\left(\frac{1}{x}\right),$$

ce qui permet de ramener le calcul dans l'intervalle $[0; 1]$, garantissant ainsi une convergence rapide.

4.9.3 Développement en série de Taylor de l'arctangente

Le développement limité de $\arctan(x)$ autour de $x = 0$ est donné par la série suivante :

$$\arctan(x) = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{2n+1} + \mathcal{O}(x^{2n+3}).$$

La série possède un rayon de convergence $R = 1$. Pour $x \in [0; 1]$, elle converge rapidement.

Nous définissons les termes successifs de cette série comme suit :

$$T_n = (-1)^n \frac{x^{2n+1}}{2n+1},$$

$$T_{n+1} = T_n \cdot -x\frac{(2n+1)}{(2n+3)}.$$

L'algorithme repose sur cette relation de récurrence. L'implémentation suit les étapes suivantes :

— On initialise la somme avec le premier terme $T_0 = x$.

— À chaque itération, on calcule T_{n+1} en utilisant la relation de récurrence et on l'ajoute à la somme.

— Le processus s'arrête soit lorsque le terme ajouté devient négligeable, soit lorsqu'un nombre maximal d'itérations est atteint.

4.9.4 Tests de Précision

Pour valider notre algorithme d'arctangente, nous procédons de manière similaire aux tests pour les autres fonctions trigonométriques inverses. Les erreurs sont définies comme suit :

$$\text{Erreur absolue} = |\text{notrearctan}(x) - \arctan_{\text{java}}(x)|$$

$$\text{Erreur relative} = \frac{|\text{notrearctan}(x) - \arctan_{\text{java}}(x)|}{\text{ulp}(\arctan_{\text{java}}(x))}.$$

Proche de 1 Notre fonction arctangente est censée renvoyer des résultats proches de 0 lorsque x est proche de 1. Cet intervalle est particulièrement sensible car la densité des flottants est élevée autour de 0, ce qui peut affecter la précision :

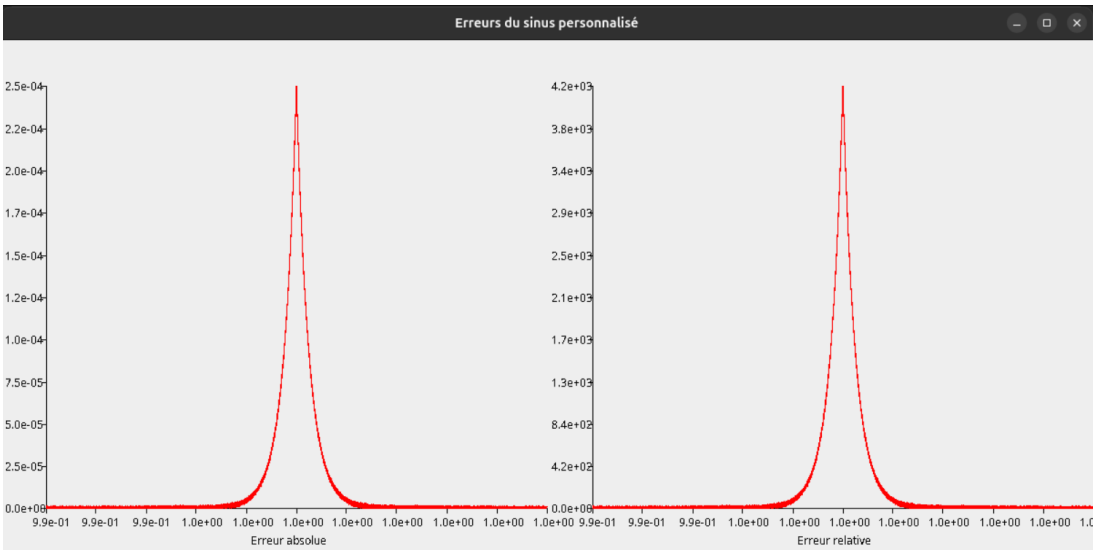


FIGURE 41 – Erreurs de l'arctangente pour x proche de 1

On observe ici une erreur absolue assez faible autour de 1 mais très grande en erreur relative ce qui nous empêche de valider le test , nous sommes précis qu'à une centaine d'ulp.

Intervalle]-1 ;1[Si on enlève les imprécision du a x=1

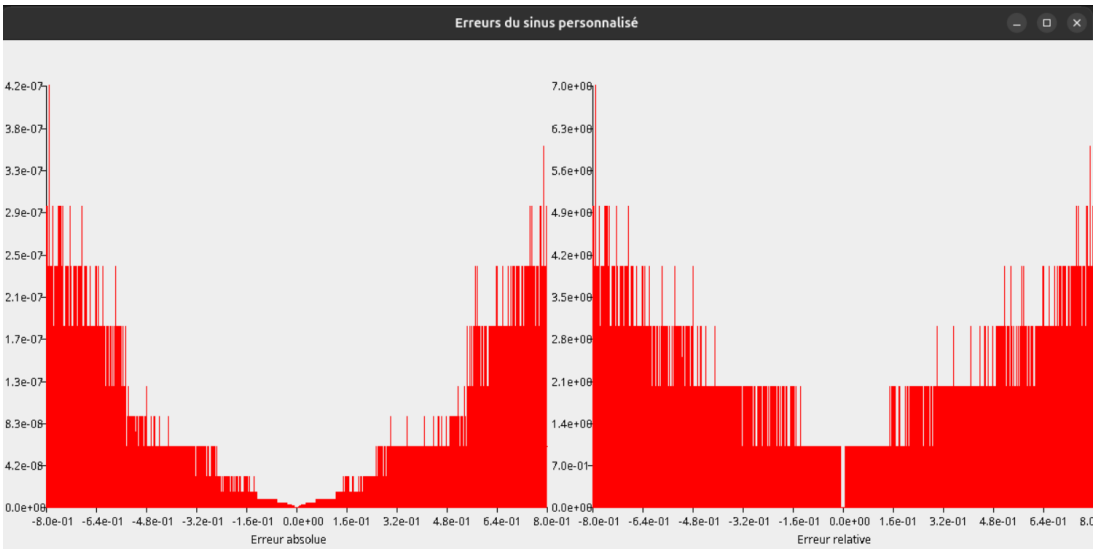


FIGURE 42 – Erreurs de l'arctangente sur]-1 ;1[

On observe ici une erreur absolue assez faible qui grandit quand on s'éloigne de 0 ce qui est logique au vu du comportement des séries de taylor. On a aussi une précision de l'ordre de quelques ulp ce qui est convenable.

Intervalle]1 ;∞[On regarde si quand on va vers l'infini notre fonction reste précise.

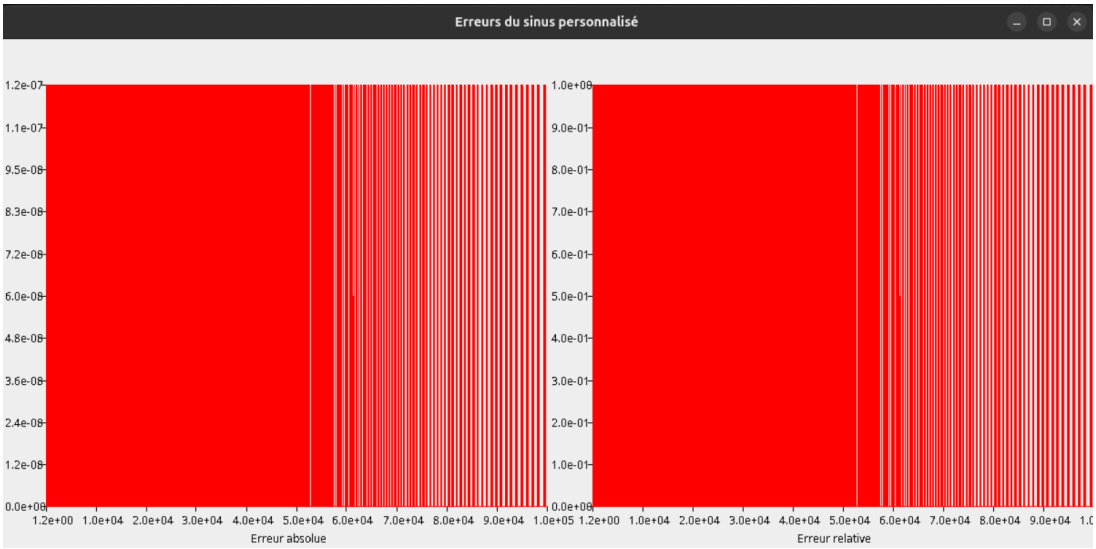


FIGURE 43 – Erreurs de l'arctangente sur]1 ;∞[

On observe ici une erreur très faible ce qui est cohérent avec notre manière de cacluler notre tangente.

4.9.5 Tests de Rapidité

Le test de rapidité se fait en observant le temps de calcul pour différents ensembles de valeurs, couvrant des intervalles variés :

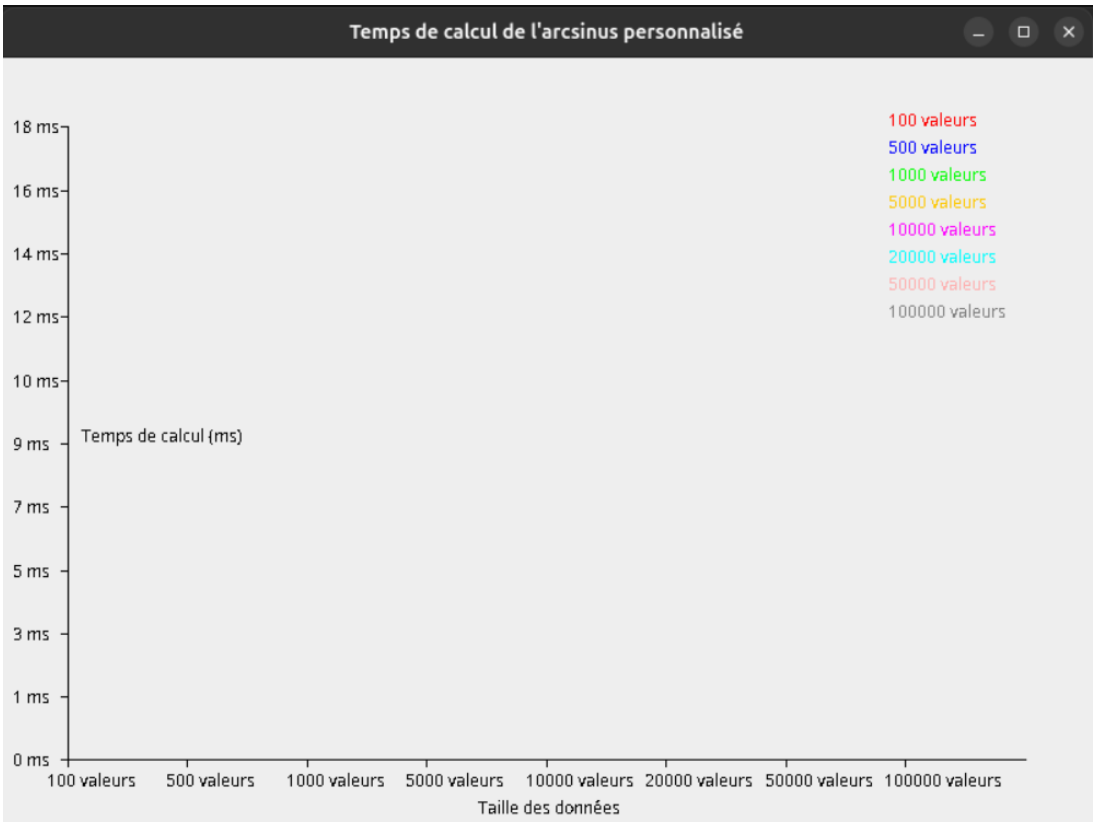


FIGURE 44 – Temps de calcul de l'arctangente pour différents ensembles de valeurs

Les résultats montrent que le temps de calcul reste rapide, même pour des ensembles volumineux de données, grâce à l'utilisation des symétries et de la relation complémentaire.

4.9.6 Analyse des tests

Les test de rapidité et de précision ne permettent malheureusement pas de valider notre fonction arctangente a cause de la borne 1. Sinon l'algoriithme est rapide et précis sur tout $\mathbb{R}$.

5 Validation des tests

Tests de rapidité :

- Cosinus
- Sinus
- Tangente
- Arccos
- Arcsin
- Arctan

Tests de précision :

- Cosinus
- Sinus

— Tangente

— Arccos : pas assez précis quand x=1

— Arcsin

— Arctan : pas assez précis quand x=1

6 Conclusion

Conclusion L'extension trigonométrique développée dans le cadre de ce projet propose un ensemble de fonctions (sinus, cosinus, tangente, arcsinus, arccosinus et arctangente) intégrées à la bibliothèque `math.deca`. Les stratégies d'implémentation se sont basées sur des méthodes classiques (séries de Taylor) et des propriétés de symétrie, complétées par une gestion fine de la réduction modulaire pour traiter la périodicité.

Les tests réalisés confirment la précision et la rapidité des approches choisies, même si des limites apparaissent lorsque l'argument dépasse un certain ordre de grandeur ou qu'il se rapproche de points singuliers (par exemple, arccos(1) ou arctan(1)). Dans l'ensemble, pour la majorité des cas d'utilisation classiques, les résultats obtenus sont fiables à quelques ULP près.

Cette extension ouvre la voie à des applications plus avancées, et pourrait être perfectionnée en affinant encore la gestion des cas extrêmes et en implémentant d'autres méthodes d'approximation pour améliorer la robustesse numérique.

7 Sources

Les références utilisées pour ce rapport sont listées ci-dessous :

1 INP GRENOBLE ENSIMAG, *Présentation sur les calculs flottants*, décembre 2024.

2 IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, 2019.

3 Jean-Michel MULLER *Elementary Functions : Algorithms and Implementation*, Birkhäuser, 2006.

4 Brook TAYLOR/Colin MCLAURIN *https ://fr.wikipedia.org/wiki/SériedeTaylor*

5 Othmane ABDELKHALEK, *https ://sites.google.com/view/abdelkhalekothmane/license/numerica analysis-m414-tp/approximation-de-tchebychev*

6 Christophe Devalland, *L'algorithme de Cordic : https ://www.apmep.fr/IMG/pdf/cordic.pdf*